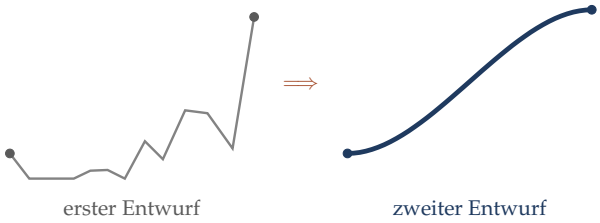
INFORMATIK VON GRUND AUF — BAND 20

Der zweite Entwurf

Programmieren in Python



Vom Lauffähigen zum Guten

Jeder Schritt erklärt. Mit vollständig entwickelten Programmen.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 20
Der zweite Entwurf · Programmieren in Python

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 20: Der zweite Entwurf.* Fassung 1.0.1.
`https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 139 Seiten.

Inhaltsverzeichnis

Vorwort 1

I Bevor man schreibt 7

1 Was die Aufgabe wirklich ist 9

2 Daten zuerst 15

3 Der Ablauf 25

II Der erste Entwurf 33

4 Der naheliegende Weg 35

5 Zerlegung 43

6 Zustand 51

III Das Handwerk 59

7 Fehlersuche als Verfahren 61

8 Fremden Code lesen 69

9 Prüfen 75

IV Python im Besonderen 81

10 Was Python anders macht 83

11 Die Werkzeuge der Sprache 89

12 Wo Python weh tut 97

V Der zweite Entwurf **103**

13 Umschreiben **105**

14 Wann aufhören **111**

15 Der Stand im Jahr 2026 **119**

VI Anhang **125**

A Regeln und Kenngrößen **127**

B Landkarte und Glossar **131**

C Kommentierte Quellen **133**

Vorwort: Warum der erste Entwurf nicht genügt

Dieses Buch setzt voraus, dass Sie programmieren können. *Es handelt nicht davon, wie man ein Programm zum Laufen bringt — das können Sie — sondern davon, was zwischen „es läuft“ und „es ist gut“ geschieht. Diese Strecke ist die Kunst, und sie steht in keinem Buch.*

Was in den Büchern fehlt

Buchart	Reicht bis	Lässt aus
Einführung	≈ 200 Zeilen	**alles Weitere**
Referenz	—	**warum**
Rezeptsammlung	Einzelfall	Zusammenhang
Musterkatalog	Lösungen	wann keine

Zwischen zweihundert und fünftausend Zeilen liegt der Bereich, in dem sich entscheidet, ob ein Programm in zwei Jahren noch änderbar ist. Genau dort hört die Einführung auf und setzt die Referenz an — und dazwischen ist nichts.

Vier Dinge, die man täglich tut und nirgends lernt.

Tätigkeit	Zeitanteil	Gelehrt?
Fremden Code lesen	≈ 30 %	**nein**
Fehler suchen	≈ 25 %	**nein**
Bestehendes ändern	≈ 25 %	kaum
Neues schreiben	≈ 20 %	ja

Die Lehre behandelt die letzte Zeile und schweigt zu den ersten drei — vier Fünftel der Arbeit. Dieses Buch dreht die Gewichtung um: Kapitel 7 und 8 behandeln Fehlersuche und Lesen als eigenständige Verfahren mit eigener Methodik.

Die Methode dieses Buches

1	Die Aufgabe wird gestellt.
2	Der **naheliegende** Weg wird gegangen — vollständig, nicht als Karikatur.
3	Er wird belastet, bis er bricht.
4	Die Ursache wird benannt.
5	**Der zweite Entwurf** entsteht — mit Begründung, was er kostet.

Schritt 2 ist der ungewöhnliche und der wichtigste. Ein Buch, das nur fertige Lösungen zeigt, vermittelt Ergebnisse; die Urteilskraft entsteht erst, wenn man gesehen hat, woran der naheliegende Weg scheitert. Deshalb steht hier viel Code, der später verworfen wird.

Das gilt auch für die Bewertung: Nicht jeder erste Entwurf ist schlecht. Bei einem Auswertungsskript, das einmal läuft, ist er die richtige Antwort. Der zweite Entwurf lohnt sich, wenn das Programm gelesen, geändert und weiterbenutzt wird — und die Entscheidung darüber gehört zur Kunst.

Das durchgehende Vorhaben

Über alle Kapitel hinweg entsteht dasselbe Programm — ein Modell der Patientenverwaltung eines Krankenhauses.

Warum dieser Fall	Er hat **echte Sonderfälle**, die man nicht erfindet
Der Gegenstand	**Befunde, Verläufe, Medikation, Warnungen** — die klinische Arbeit
Der Rahmen	Aufnahme, Fall, Schnittstellen — nötig, aber nicht das Thema
Was er nicht ist	**Kein Produkt — ein Modell zum Lernen**
Die Daten	**ausnahmslos erzeugt, niemals echt**

Die dritte Zeile ist eine Selbstbeschränkung mit Grund: Ein vollständiges System zu bauen wäre ein anderes Vorhaben und würde die Didaktik erschlagen. Wir bauen die Teile, an denen sich etwas zeigen lässt.

Die Sonderfälle, an denen Entwürfe zerbrechen.

Fall	Häufigkeit	Bricht
Ein Befund hat **vier** Zeitpunkte	immer	**ein Zeitstempel**
Nachträglich validierter Wert	täglich	Verlauf ändert sich rückwirkend
Labor wechselt die Methode	jährlich	**Referenzbereich je Wert**
Wert ohne Einheit	regelmäßig	**Zahl statt Größe**
Medikament pausiert	täglich	**aktuell/nicht aktuell**
Notfall ohne Daten	täglich	Pflichtfelder
Namensänderung	regelmäßig	Kennung = Name
Hundert Warnungen zugleich	täglich	**Rangfolge fehlt**

Jede dieser Zeilen ist ein Kapitelinhalt. Man erfindet solche Fälle nicht am Schreibtisch — man erlebt sie. Und jeder von ihnen bringt einen naheliegenden Entwurf zum Einsturz, der sonst tadellos aussieht.

Die ersten fünf Zeilen sind der eigentliche Gegenstand. Ein Krankenhausinformationssystem ist keine Verwaltung mit medizinischem Anstrich — die anspruchsvollen Probleme liegen dort, wo aus Messwerten Verläufe und aus Verläufen Entscheidungen werden.

Was dieses Buch will

Erstens: den Zwischenbereich füllen.
Von zweihundert auf fünftausend Zeilen ändern sich die Regeln. Was bei zweihundert Zeilen gleichgültig ist — Namensgebung, Zerlegung, Zustandsführung — entscheidet bei fünftausend über die Änderbarkeit.

Zweitens: die Begründung mitliefern.

Zu jeder Empfehlung gehört, was sie kostet und wann sie nicht gilt. Eine Regel ohne Ausnahmebedingung ist ein Dogma, und Dogmen erzeugen schlechten Code von der anderen Sorte.

Drittens: Python ernst nehmen.
Die Sprache hat eigene Ausdrucksmittel, und wer sie ignoriert, schreibt in Python ein Programm, das genauso gut in einer anderen Sprache stünde. Teil IV behandelt, was Python anders macht — und wo es weh tut.

Drei Warnungen

Erstens: Es gibt keine besten Praktiken.

> *Jede Regel dieses Buches hat einen Anwendungsbereich. „Funktionen sollen kurz sein" ist richtig, bis man dieselbe Logik über sechs Funktionen verstreut hat und keine mehr versteht. Die Frage ist nie, ob eine Regel gilt, sondern wann.*

Zweitens: Lesbarkeit ist keine Geschmacksfrage.
Sie ist messbar — über die Zeit, die jemand braucht, um eine Frage zu beantworten. Kapitel 8 führt das vor.
Drittens: Der zweite Entwurf ist nicht der letzte.

> *Es gibt einen dritten und einen vierten, und irgendwann hört man auf. Wann, ist die schwierigste Entscheidung dieses Fachs — Kapitel 14 behandelt sie.*

Zum Aufbau

- **Teil I** (Kap. 1–3): Bevor man schreibt. Aufgabe, **Daten**, Ablauf.
- **Teil II** (Kap. 4–6): Der erste Entwurf. Der naheliegende Weg, Zerlegung, **Zustand**.
- **Teil III** (Kap. 7–9): Das Handwerk. **Fehlersuche**, **Lesen**, Prüfen.
- **Teil IV** (Kap. 10–12): Python im Besonderen. Idiome, Werkzeuge, **Grenzen**.
- **Teil V** (Kap. 13–15): Der zweite Entwurf. Umschreiben, **wann aufhören**, der Stand 2026.

Ein Feld namens `name`

Ein Krankenhausinformationssystem, das seit über zwanzig Jahren läuft. In der Patiententabelle steht ein Feld für den Namen.

Der Entwurf	Ein Textfeld, 60 Zeichen, „Nachname, Vorname"
Der Grund	So steht es auf dem Formular
Die Annahme	**Ein Mensch hat einen Namen, und der ändert sich nicht**

Die Annahme ist falsch, und zwar auf mehrere Weisen gleichzeitig. Sie ist trotzdem verständlich: Sie bildet ab, was auf dem Papierformular stand — und das Papierformular war seit vierzig Jahren in Gebrauch, ohne aufzufallen.

Was daraus wurde.

Fall	Behelf	Folge
Namens-änderung	überschreiben	**Altbefunde nicht auffindbar**
Doppelnamen	Komma im Namen	Zerlegung bricht
Titel	vorangestellt	**Sortierung falsch**
Zu lang	abgeschnitten	stille Datenver-fälschung
Sonderzeichen	ersetzt	Suche findet nicht

Fünf Behelfe, jeder für sich vernünftig, jeder eine neue Fehlerquelle. Und keiner von ihnen ist rückgängig zu machen, weil die ursprüngliche Information nicht mehr existiert — die vierte Zeile ist der schlimmste Fall.

Kapitel 2 nimmt diesen Fall auseinander und entwickelt den zweiten Entwurf. Die Lehre vorweg: Die teuerste Entscheidung eines Programms ist das Datenmodell, sie fällt am frühesten, und man trifft sie mit dem geringsten Wissen.

Den ersten Entwurf schreibt man für die Maschine.
Er muss laufen.
Den zweiten schreibt man für den Menschen,
der ihn in drei Jahren ändern muss —
und der wird man selbst sein.

— der Leitgedanke dieses Bandes.

Part I

Bevor man schreibt

1 Was die Aufgabe wirklich ist

Die Anforderung ist nicht die Aufgabe

Die Anforderung	„Zeigen Sie die Laborwerte des Patienten an“
Die Aufgabe	**Der Arzt will wissen, ob sich etwas verschlechtert hat**
Der Unterschied	Eine Tabelle erfüllt die Anforderung und verfehlt die Aufgabe

Wer die Anforderung umsetzt, baut eine Liste von Zahlen. Wer die Aufgabe versteht, baut eine Darstellung, in der eine Verschlechterung ins Auge springt. Beides erfüllt das Pflichtenheft; nur eines ist brauchbar.

Die Frage, die weiterhilft.

Nicht	„Was soll das Programm tun?“
Sondern	**„Welche Entscheidung soll es ermöglichen?“**
Und dann	Welche Angabe braucht diese Entscheidung — und welche nicht?

Die zweite Zeile ist die nützlichste Frage dieses Kapitels. Sie führt fast immer zu einem kleineren und besseren Programm, weil sie offenlegt, welche Hälfte der geforderten Angaben nie jemand ansieht.

Der Fall: eine Befundübersicht

Die Anforderung, wie sie ankommt.

> *„Wir brauchen eine Übersicht der Laborwerte. Alle Parameter, chronologisch, mit Referenzbereich. Auffällige Werte sollen markiert werden. Export nach Excel.“*

Die Fragen, die niemand gestellt hat.

Frage	Beantwortet?	Folge
Welcher Zeitpunkt zählt?	**nein**	**Kap. 2**
Auffällig heißt was?	nein	außerhalb? oder verändert?
Wie viele Werte?	nein	20 oder 8000
Was tut der Arzt damit?	**nein**	**die Kernfrage**
Wozu der Export?	nein	**meist ein Symptom**

Die letzte Zeile ist die aufschlussreichste. Ein gewünschter Export nach Excel bedeutet fast immer, dass die Anwendung eine Aufgabe nicht erfüllt und die Anwender sie außerhalb erledigen. Wer den Export baut, zementiert das Problem; wer fragt, wozu, findet die eigentliche Anforderung.

Was das Nachfragen ergab.

Der Export	Diente dazu, zwei Werte über die Zeit *nebeneinander* zu sehen
Auffällig	Nicht „außerhalb des Bereichs“, sondern **„anders als letzte Woche“**
Die Menge	Bei einem Intensivpatienten **mehrere tausend Werte**

Damit ist es eine andere Aufgabe. Nicht eine markierte Tabelle, sondern eine Verlaufsdarstellung mit wählbaren Parametern — und eine Auffälligkeitsdefinition, die Veränderung misst statt Grenzüberschreitung.

Warum „außerhalb des Bereichs“ zu wenig ist

Tag	Wert	Bereich	Bewertung
1	138	135–145	normal
3	136	135–145	normal
5	135	135–145	normal
7	**133**	135–145	auffällig

Eine Markierung erscheint an Tag sieben. Interessant war der Verlauf ab Tag eins — ein stetiger Abfall innerhalb des Normbereichs. Eine Darstellung, die nur Grenzüberschreitungen hervorhebt, zeigt die Information zuletzt, wenn sie am wenigsten wert ist.

Aufgabe 1.1.

(a) Zerlegen Sie eine Anforderung in Entscheidungen.
(b) Bestimmen Sie, welche Angaben tatsächlich gebraucht werden.
(c) Rechnen Sie die Kosten einer verfehlten Aufgabe aus.

Lösung Schritt für Schritt.

(a) Anforderung in Entscheidungen zerlegen.
Das Verfahren. Für jede geforderte Anzeige: Welche Entscheidung hängt daran?

Angabe	Entscheidung	Nötig?
Aktueller Wert	Handeln oder nicht	**ja**
Referenzbereich	Einordnung	ja
Verlauf	**Richtung erkennen**	**ja**
Analysemethode	Vergleichbarkeit	bei Wechsel
Auftragsnummer	keine	nein
Eingebender Benutzer	keine	nur im Zweifel
Gerätekennung	keine	nein

Drei von sieben Angaben tragen Entscheidungen. Die übrigen sind nicht wertlos — sie werden bei Rückfragen gebraucht — aber sie gehören nicht in die Hauptansicht. Das ist die Anwendung von Band 18, Kapitel 1: Was ständig angezeigt wird, verlängert die Suchzeit für alles andere.

Die Wirkung auf die Suchzeit.

Spalten	Suchzeit	bei 40 Zeilen	
7	2,8 s	2,8 s	
4	1,9 s	1,9 s	
3	**1,5 s**	1,5 s	−46 %

Bei zwanzig Aufrufen je Visite und zweihundert Visiten im Jahr sind das gut anderthalb Stunden — für das Weglassen von vier Spalten. Weglassen ist die billigste Verbesserung, die es gibt, und die am seltensten vorgeschlagene.

(b) Welche Angaben gebraucht werden.
Die Probe. Man beobachtet, was tatsächlich benutzt wird.

Angabe	Angezeigt	Angesehen	Quote
Wert	100 %	100 %	1,00
Referenzbereich	100 %	68 %	0,68
Verlauf	0 %	**gesucht**	—
Auftragsnummer	100 %	**2** %	**0,02**
Gerätekennung	100 %	0 %	0,00

Die dritte Zeile ist der eigentliche Befund: Was gebraucht wird, ist gar nicht da — die Anwender stellen es sich im Tabellenprogramm selbst her. Die letzten beiden Zeilen sind ständig da und werden nie angesehen.

Diese Erhebung kostet einen Nachmittag mit einem Anwender und beantwortet mehr Fragen als jedes Pflichtenheft. Band 18, Kapitel 11 behandelt das Verfahren; hier genügt der Grundsatz: Man sieht zu, statt zu fragen.

(c) Kosten einer verfehlten Aufgabe.
Die Rechnung. Der Export als Behelf.

Posten	Wert	
Vorgänge je Tag	14	je Station
Zeit je Vorgang	3,5 min	Export, Umformen, Bild
Je Tag und Station	49 min	
8 Stationen, 250 Tage	**1633 h/Jahr**	
Verlaufsansicht bauen	≈ 160 h	einmalig
Amortisation	≈ **5 Wochen**	

Fünf Wochen — und die Verlaufsansicht wurde nicht gebaut, weil sie nicht im Pflichtenheft stand. Der Export stand drin, war in zwei Tagen fertig und hat das Problem für zwölf Jahre festgeschrieben.

Der versteckte Posten.

Folge des Exports	Bewertung	
Daten verlassen das System	**ungeschützt**	Band 19
Stand veraltet sofort	Fehlerquelle	
Umformung von Hand	Übertragungsfehler	
Keine Nachvollziehbarkeit	**keine Aufzeichnung**	

Ein Behelf, der außerhalb des Systems stattfindet, verlässt zugleich alle Sicherungen, die im System gelten. Das ist Saltzers achter Grundsatz aus Band 19: Was zu umständlich ist, wird umgangen — und die Umgehung ist ungeschützt.

Der Kunde beschreibt Lösungen, nicht Probleme

Was gesagt wird	„Wir brauchen einen Export nach Excel“
Was gemeint ist	**„Ich kann zwei Verläufe nicht nebeneinander sehen“**
Die Frage	„Was machen Sie damit, wenn Sie es haben?“

Anwender formulieren ihre Wünsche im Rahmen dessen, was sie für machbar halten — und das ist meist das, was sie schon kennen. Die dritte Zeile ist die einzige Frage, die man braucht, und sie funktioniert fast immer.

Das ist keine Bevormundung. Der Anwender kennt seine Aufgabe besser als jeder Entwickler; er kennt nur die Lösungsmöglichkeiten nicht. Die Arbeitsteilung ist klar — und sie funktioniert nur, wenn man nach der Aufgabe fragt statt nach der Lösung.

Bentley[4] zeigt an durchgerechneten Fällen, wie das Verstehen der Aufgabe das Programm bestimmt; Hunt und Thomas[10] behandeln das Nachfragen als Handwerkszeug; Ousterhout[19] begründet, warum das Weglassen die wirksamste Entwurfsentscheidung ist.

2 Daten zuerst

Die teuerste Entscheidung

Entscheidung	Fällt	Rücknahme-kosten
Datenmodell	**zuerst**	**Jahre**
Zerlegung	früh	Wochen
Verfahren	unterwegs	Stunden
Namensgebung	laufend	Minuten

Die erste Zeile ist die einzige, die man nicht korrigieren kann, sobald Daten darin liegen. Sie fällt zu dem Zeitpunkt, an dem man am wenigsten über die Aufgabe weiß — das ist die Grundschwierigkeit dieses Kapitels.

Wirths Formel.

$$\text{Algorithmen} + \text{Datenstrukturen} = \text{Programme}$$

Der Titel von 1976 ist eine Gleichung mit einer Rangordnung darin: Wer die Datenstruktur richtig wählt, bekommt das Verfahren fast geschenkt. Wer sie falsch wählt, kämpft in jedem Verfahren dagegen an.

Der erste Entwurf: ein Messwert

Der naheliegende Weg.

```
{
"patient":  "12345",
"test":  "Kalium",
"wert":  4.2,
"zeit":  "2026-03-14T08:30:00",
"min":  3.5, "max":  5.1
}
```

Das ist ein vernünftiger Entwurf. Er ist lesbar, er deckt den Regelfall, und er würde in jeder Einführung so stehen. Wir belasten ihn jetzt mit sechs Fällen aus dem Alltag.

Die Belastungsprobe.

Fall	Was fehlt	Folge
Welche Zeit ist das?	**Abnahme? Messung?**	**Verlauf falsch**
Einheit	`mmol/l`?	**Zahl ohne Bedeutung**
Wert „< 0,1“	**nur Zahlen möglich**	Wert geht verloren
Hämolytisch	Qualitäts-vermerk	Wert unbrauchbar, gilt als gültig
Nachträglich korrigiert	keine Fassung	alter Wert weg
Labor wechselt Methode	Bereich am Wert	alte Werte falsch bewertet

Sechs Fälle, sechs Brüche — und keiner davon ist exotisch. Die zweite Zeile ist die peinlichste: Ein Zahlenwert ohne Einheit ist keine Größe, und das gilt seit Newton. Die Marssonde von 1999 ist genau daran zerbrochen.

Die vier Zeiten eines Befundes

Zeitpunkt	Wofür	Reihenfolge
Abnahme	**klinische Deutung**	1
Messung	Methodenfragen	2
Validierung	Freigabe durch Labor	3
Sichtung	**ärztliche Kenntnis**	4 — oder nie

> *Für den Verlauf zählt die Abnahmezeit — sie sagt, wann der Zustand vorlag. Für die Frage „wusste jemand davon“ zählt die Sichtung. Ein System mit einem Zeitstempel kann eine der beiden Fragen nicht beantworten, und meistens ist es die zweite.*

Die Reihenfolge ist auch nicht garantiert: Ein nachträglich beauftragter Test wird nach der Abnahme angeordnet, ein korrigierter Wert wird nach der ersten Sichtung validiert. Wer die Ordnung annimmt, baut Sortierungen, die gelegentlich falsch sind.

Der zweite Entwurf

```
@dataclass(frozen=True)
class Messgroesse:
code:  str          # LOINC o. ä.
einheit:  str

@dataclass(frozen=True)
class Befund:
patient:  PatientId
groesse:  Messgroesse
wert:  Ergebnis      # Zahl ODER Grenzangabe
abnahme:  datetime
validierung:  datetime | None
bereich:  Referenzbereich
qualitaet:  frozenset[Vermerk]
ersetzt:  BefundId | None
```

Vier Entscheidungen stecken darin. `frozen` *macht den Befund unveränderlich — eine Korrektur ist ein neuer Befund mit Verweis, kein Überschreiben.* `einheit` *sitzt an der Größe, nicht am Wert.* `bereich` *liegt* am Befund, *nicht in einer Tabelle. Und* `wert` *ist ein eigener Typ, weil „< 0,1“ keine Zahl ist.*

Warum der Bereich am Befund hängt.

	Bereich in Tabelle	**Bereich am Befund**
Speicher	sparsam	redundant
Methodenwechsel	**alte Werte falsch**	**bleiben richtig**
Nachvollziehbar	nein	ja

Die Redundanz ist hier kein Fehler, sondern der Zweck: Der Bereich ist Teil dessen, was damals galt. Wer ihn nachschlägt, bewertet historische Werte nach heutigen Maßstäben — und das ist bei jedem Methodenwechsel falsch.

Der Fall `name` aus dem Vorwort

	Erster Entwurf	**Zweiter Entwurf**
Form	**ein Textfeld**	**Namensteile, benannt**
Änderung	überschreiben	neue Fassung, gültig ab
Suche	über die aktuelle Form	**über alle Fassungen**
Kennung	implizit der Name	eigene, stabile Kennung

Die dritte Zeile ist der praktische Gewinn: Nach einer Namensänderung findet man die Altbefunde weiterhin, weil die frühere Fassung erhalten bleibt. Die vierte ist die grundsätzliche Lehre — ein fachlicher Wert taugt nie als Kennung, weil fachliche Werte sich ändern.

Aufgabe 2.1.

(a) Entwerfen Sie den Ergebnistyp.
(b) Modellieren Sie die Medikation über die Zeit.
(c) Rechnen Sie die Kosten der Unveränderlichkeit aus.

Lösung Schritt für Schritt.

(a) Der Ergebnistyp.
Das Problem. Ein Laborergebnis ist nicht immer eine Zahl.

Erscheinung	Beispiel	Als Zahl?
Messwert	4,2	ja
Unter Nachweisgrenze	`< 0.1`	**nein**
Über Messbereich	`> 1000`	nein
Qualitativ	`positiv`	nein
Nicht auswertbar	`Probe hämolytisch`	nein

Der naheliegende Behelf und sein Preis.

Behelf	Wirkung	
Als Text speichern	nicht rechenbar	**kein Verlauf**
`< 0.1` als 0,1	**falsch**	stille Verfälschung
`< 0.1` als 0	falsch	
`None`	Information weg	

Der zweite Entwurf.

```
class Ergebnis:  ...

@dataclass(frozen=True)
class Zahlwert(Ergebnis):
betrag:  Decimal

@dataclass(frozen=True)
class Grenzwert(Ergebnis):
richtung:  Literal["<", ">"]
schranke:  Decimal

@dataclass(frozen=True)
class Qualitativ(Ergebnis):
befund:  str

@dataclass(frozen=True)
class Nichtauswertbar(Ergebnis):
grund:  str
```

Vier Fälle, vier Typen. Der Gewinn liegt nicht in der Eleganz, sondern darin, dass jede Auswertung sich nun entscheiden *muss: Wer einen Mittelwert über Befunde bildet, muss angeben, was mit Grenzwerten geschieht. Vorher hat das Programm stillschweigend etwas Falsches getan.*

Warum `Decimal` und nicht `float`: Laborwerte kommen mit einer festgelegten Stellenzahl, und $0{,}1 + 0{,}2$ *ist in Gleitkomma nicht* $0{,}3$*. Band 0, Kapitel 3 erklärt es; hier genügt die Regel — wo gerundete Dezimalzahlen fachlich bedeutsam sind, kein Gleitkomma.*

(b) Medikation über die Zeit.
Der erste Entwurf. Eine Liste aktueller Medikamente je Patient.

Frage	Beantwortbar?	
Was nimmt er jetzt?	ja	
Was nahm er am 3. März?	**nein**	**Historie fehlt**
Wann wurde umgestellt?	nein	
Warum abgesetzt?	nein	
Pausiert oder abgesetzt?	**nein**	ein Zustand zu wenig

Der zweite Entwurf: Anordnungen mit Gültigkeit.

```
@dataclass(frozen=True)
class Anordnung:
wirkstoff:  Wirkstoff
dosis:  Dosis
ab:  datetime
bis:  datetime | None   # offen = laufend
grund:  str
ende_grund:  Endgrund | None
ersetzt:  AnordnungId | None
```

Aus einer Liste wird eine Menge von Zeitabschnitten. Die aktuelle Medikation ist dann kein gespeicherter Zustand, sondern eine Abfrage*: alle Anordnungen, deren Gültigkeit den gewünschten Zeitpunkt enthält.*

```
def medikation_am(anordnungen, zeitpunkt):
return [a for a in anordnungen
if a.ab <= zeitpunkt
and (a.bis is None or zeitpunkt < a.bis)]
```

Vier Zeilen — und sie beantworten alle fünf Fragen aus der Tabelle oben. Das ist Wirths Rangordnung in Reinform: Die richtige Datenstruktur macht das Verfahren trivial.

Die Unterscheidung, die man leicht übersieht.

Zustand	Modellierung	Bedeutung
Laufend	`bis is None`	
Beendet	`bis` gesetzt	**mit `ende_grund`**
Pausiert	**beendet + Folgeanordnung**	Lücke im Verlauf
Umgestellt	beendet + neue mit `ersetzt`	Kette bleibt

Pausieren braucht keinen eigenen Zustand — es ist eine Lücke zwischen zwei Abschnitten. Wer dafür ein Kennzeichen einführt, hat zwei Wahrheiten über denselben Sachverhalt und muss sie synchron halten.

(c) Kosten der Unveränderlichkeit.

Größe	Ändernd	Unveränderlich	Faktor
Speicher je Patient	1,0	≈ 2,4	2,4
Schreibzugriffe	1,0	1,0	1,0
Lesen „aktuell“	1,0	≈ 1,3	1,3
Lesen „historisch“	unmöglich	≈ 1,3	—
Fehlerklassen	**Verlust, Wettlauf**	**keine**	

Zweieinhalbfacher Speicher und dreißig Prozent langsamere Lesezugriffe — bei heutigen Größenordnungen belanglos. Dafür verschwinden zwei ganze Fehlerklassen: verlorene Änderungen und Wettläufe beim gleichzeitigen Schreiben. Kapitel 6 führt das aus.

Wo die Rechnung kippt.

Fall	Unveränderlich?	
Klinische Daten	**unbedingt**	**rechtlich gefordert**
Stammdaten	ja	Namensänderung
Sitzungszustand	nein	kurzlebig
Messreihe, 10^8 Punkte	nein	Speicher

Die letzte Zeile ist die Ausnahme, die die Regel bestätigt: Bei sehr großen Datenmengen kippt die Rechnung. Bei allem, was ein Mensch angeordnet oder befundet hat, kippt sie nicht — und dort ist die Nachvollziehbarkeit ohnehin vorgeschrieben.

Das Datenmodell zuerst, aber nicht endgültig

Der Widerspruch	Man entscheidet am frühesten mit dem geringsten Wissen
Der falsche Schluss	Also lange planen, bevor man anfängt
Der richtige	**Früh ein einfaches Modell, früh mit echten Fällen belasten**

Kein Nachdenken am Schreibtisch hätte die sechs Fälle aus der Belastungsprobe hervorgebracht. Sie kommen aus dem Alltag — und man findet sie, indem man das Modell früh an echten Daten prüft, nicht indem man länger überlegt.

Die praktikable Reihenfolge lautet deshalb: ein Modell, das den Regelfall trägt; dann zwanzig echte Fälle dagegenhalten; dann der zweite Entwurf. Vor der ersten Zeile Anwendungscode — danach wird es teuer.

Wirth[27] stellte die Rangordnung von Datenstruktur und Verfahren in den Titel; Parnas[20] begründete, warum das Verbergen der Darstellung die tragende Entwurfsentscheidung ist; Ramalho[22] zeigt die Ausdrucksmittel, mit denen sich solche Modelle in Python fassen lassen.

3 Der Ablauf

Was ein Ablauf leisten muss

Richtig rechnen	Selbstverständlich — und die niedrigste Hürde
Lesbar sein	**Man sieht, warum es richtig ist**
Enden	Wird selten geprüft und ist ein eigener Beweis

Die zweite Zeile ist der Gegenstand dieses Kapitels. Ein Ablauf, dessen Richtigkeit man nur durch Nachvollziehen jedes Falles erkennt, ist nicht wartbar — man muss ihn jedes Mal neu verstehen. Ein guter Ablauf trägt seine Begründung mit sich.

Die Invariante als Lesehilfe.

```
def letzter_vor(befunde, zeitpunkt):
"""befunde ist nach abnahme aufsteigend
sortiert."""
treffer = None
for b in befunde:
# treffer ist der letzte Befund vor zeitpunkt
# unter den bisher gesehenen
if b.abnahme >= zeitpunkt:
break
treffer = b
return treffer
```

Der Kommentar in der Schleife ist keine Erklärung des Codes — der ist offensichtlich — sondern die Aussage, die vor und nach jedem Durchlauf gilt. Wer sie liest, muss die Schleife nicht mehr im Kopf durchspielen. Band 17, Kapitel 9 nennt es Invariante; hier ist es ein Lesewerkzeug.

Der erste Entwurf: Verlauf zusammenstellen

Die Aufgabe. Zu einer Messgröße die Werte der letzten n Tage, aufsteigend, ohne ersetzte Fassungen.

```
def verlauf(befunde, patient, code, tage):
ergebnis = []
grenze = datetime.now() - timedelta(days=tage)
for b in befunde:
if b.patient == patient:
if b.groesse.code == code:
if b.abnahme > grenze:
ist_ersetzt = False
for c in befunde:
if c.ersetzt == b.id:
ist_ersetzt = True
if not ist_ersetzt:
ergebnis.append(b)
ergebnis.sort(key=lambda b:  b.abnahme)
return ergebnis
```

Das Programm ist richtig. Es tut genau das, was verlangt war, und würde jede Prüfung bestehen. Es hat drei Mängel, und alle drei sind bei fünfzig Befunden unsichtbar.

Die drei Mängel.

Mangel	Wirkung	Sichtbar ab
Vierfache Schachtelung	**schwer zu lesen**	sofort
Innere Schleife	$\mathcal{O}(n^2)$	$\approx$ 5000 Befunde
`now()` innen	**nicht prüfbar**	beim ersten Test

Die dritte Zeile ist die folgenreichste und die am leichtesten zu behebende. Eine Funktion, die die Uhrzeit selbst holt, liefert bei jedem Aufruf ein anderes Ergebnis — man kann sie nicht prüfen, ohne die Zeit zu manipulieren.

Zeit als Argument

	`now()` innen	Zeit als Argument
Prüfbar	nur mit Kunstgriff	**unmittelbar**
Historische Sicht	unmöglich	**fällt ab**
Aufrufer merkt	nichts	ein Argument mehr

Die zweite Zeile ist der unerwartete Gewinn: Wer die Zeit hereinreicht, bekommt die Frage „wie sah der Verlauf am 3. März aus“ geschenkt — und genau die wird in der Klinik ständig gestellt, etwa bei der Nachbesprechung eines Falles.

Die Regel dahinter ist allgemein: Alles, was eine Funktion aus der Umgebung zieht — Uhrzeit, Zufall, Dateisystem, Netz — macht sie unprüfbar. Wer es hereinreicht, gewinnt Prüfbarkeit und meist noch eine Fähigkeit dazu.

Aufgabe 3.1.

(a) Entwickeln Sie den zweiten Entwurf.
(b) Messen Sie den Unterschied.
(c) Prüfen Sie die Terminierung einer Schleife.

Lösung Schritt für Schritt.

(a) Der zweite Entwurf.
Schritt 1: Die Schachtelung auflösen. Bedingungen früh verlassen statt tief schachteln.

```
def verlauf(befunde, patient, code, ab, bis):
ersetzte = {b.ersetzt for b in befunde if
b.ersetzt}
treffer = [
b for b in befunde
if b.patient == patient
and b.groesse.code == code
and ab <= b.abnahme < bis
and b.id not in ersetzte
]
return sorted(treffer, key=lambda b:  b.abnahme)
```

Drei Änderungen. Die Menge der ersetzten Kennungen wird einmal *gebildet statt für jeden Befund neu — damit fällt die innere Schleife weg. Die Bedingungen stehen nebeneinander statt geschachtelt. Und statt* `tage` *nimmt die Funktion einen Zeitraum, womit auch die Rückschau möglich wird.*

Die Bilanz.

Größe	Erster	Zweiter	
Zeilen	16	**10**	
Schachtelungstiefe	5	**2**	lesbar
Aufwand	$\mathcal{O}(n^2)$	$\mathcal{O}(\mathbf{n} \log \mathbf{n})$	
Prüfbar	nein	**ja**	
Rückschau	nein	ja	

Die letzte Zeile war nicht gefordert. Sie fällt ab, weil die Zeit zum Argument wurde — ein wiederkehrendes Muster: Der zweite Entwurf löst das genannte Problem und beantwortet nebenbei eine Frage, die noch niemand gestellt hat.

(b) Den Unterschied messen.

Befunde	Erster	Zweiter	Faktor
500	12 ms	0,4 ms	30
5 000	1,1 s	4 ms	275
50 000	**112 s**	**48 ms**	**2330**
500 000	≈ 3 h	0,6 s	

Bei fünfhundert Befunden ist der Unterschied belanglos — zwölf Millisekunden merkt niemand. Bei fünfzigtausend, also einem Intensivpatienten nach mehreren Wochen, ist die erste Fassung unbenutzbar. Genau dort brechen Systeme: nicht im Regelfall, sondern beim schwersten Fall.

Die Lehre für die Praxis.

Regel	**Gilt?**	
Immer optimieren	nein	verschwendet Zeit
Nie optimieren	nein	
Größenordnung kennen	**ja**	**vor dem Schreiben**
Messen statt raten	ja	Kapitel 12

Man muss nicht jedes Programm optimieren, aber man muss wissen, mit welchen Größenordnungen man rechnet. Die Frage „wie viele Befunde hat ein Patient im schlimmsten Fall“ kostet eine Minute und entscheidet über die Wahl des Verfahrens.

(c) Terminierung prüfen.
Der Fall. Zusammenführen zweier sortierter Messreihen.

```
i = j = 0
while i < len(a) or j < len(b):
if j >= len(b) or (i < len(a) and a[i].t <=
b[j].t):
nimm(a[i]); i += 1
else:
nimm(b[j]); j += 1
```

Das Terminierungsmaß.

Bedingung	Erfüllt?	
Maß: $(\lvert a\rvert - i) + (\lvert b\rvert - j)$	—	
Ganzzahlig	ja	
Nach unten beschränkt	≥ 0	
Nimmt je Durchlauf ab	**um genau** 1	**terminiert**

Der Beweis ist drei Zeilen lang und deckt zugleich einen möglichen Fehler auf: Gäbe es einen Zweig, der weder `i` noch `j` erhöht, würde das Maß nicht abnehmen — und die Schleife liefe ewig. Genau so entstehen Endlosschleifen in Zusammenführungen.

Wo es schiefgeht.

Fehlerbild	Ursache	
Endlosschleife	Zweig ohne Fortschritt	**Maß nimmt nicht ab**
Ein Element fehlt	Abbruch zu früh	`and` statt `or`
Bereichsfehler	Prüfung fehlt	`b[j]` bei $j = \lvert b\rvert$

Alle drei entstehen an derselben Stelle — der Abbruchbedingung — und alle drei werden von einem Terminierungsmaß oder einer Invariante aufgedeckt. Das ist der praktische Nutzen der Formalismen aus Band 17: nicht Beweise, sondern Fehlersuche vor dem Schreiben.

Frühes Verlassen gegen tiefe Schachtelung

	Schachtelung	Frühes Verlassen
Zustand im Kopf	**alle Bedingungen**	**nur die aktuelle**
Neue Bedingung	tiefer schachteln	eine Zeile anhängen
Lesbar bis	≈ 3 Ebenen	beliebig

Die erste Zeile ist die Begründung und stammt aus Band 18: Das Arbeitsgedächtnis fasst etwa vier Einheiten. Bei fünf Schachtelungsebenen muss man fünf Bedingungen gleichzeitig im Kopf halten — das ist über der Grenze, und deshalb liest sich solcher Code so mühsam.

Das ist keine Stilfrage. Die Lesbarkeitsgrenze ist eine Eigenschaft des Lesers, nicht des Codes — und sie liegt für alle Menschen ungefähr gleich.

Dijkstra[6] entwickelte das Programmieren aus Invariante und Terminierungsmaß; Abelson und Sussman[1] zeigen, wie sich Abläufe aus wenigen Bausteinen aufbauen lassen; Kernighan und Plauger[12] formulierten die Regeln für lesbaren Ablauf bereits 1978.

Part II

Der erste Entwurf

4 Der naheliegende Weg

Wann der erste Entwurf genügt

Programm	Zweiter Entwurf?	Grund
Einmalige Auswertung	**nein**	**wird nie gelesen**
Prototyp zur Klärung	nein	aber: Kap. 14
Skript für die Schublade	nein	
Wird von anderen benutzt	**ja**	
Läuft länger als ein Jahr	**ja**	
Trägt Entscheidungen	unbedingt	**klinisch**

Der zweite Entwurf ist kein Selbstzweck. Bei einem Auswertungsskript, das einmal läuft und dann verschwindet, ist er verschwendete Zeit — und die Fähigkeit, das zu erkennen, gehört zur Kunst genauso wie die Fähigkeit, ihn zu schreiben.

Die Falle steht in Zeile zwei und ist aus Band 17 bekannt: Der Prototyp geht in Betrieb, weil er funktioniert. Wer einen baut, sollte entweder sicherstellen können, dass er verschwindet — oder ihn so bauen, als bliebe er.

Der Fall: Auffälligkeiten erkennen

Die Aufgabe. Aus einem Verlauf die Werte hervorheben, die eine Reaktion verlangen.
Der erste Entwurf.

```
def auffaellig(b):
if b.wert.betrag < b.bereich.min:
return True
if b.wert.betrag > b.bereich.max:
return True
return False
```

Sechs Zeilen, offensichtlich richtig, und sie beantworten die falsche Frage. Wir belasten sie mit fünf Fällen, die alle täglich vorkommen.

Fall	Verhalten	Richtig?
`Grenzwert("<", 0.1)`	**Absturz**	kein `betrag`
Chronisch erhöht, stabil	markiert	**Rauschen**
Abfall im Normbereich	nicht markiert	**das Wichtige fehlt**
Hämolytische Probe	markiert	Wert ist ungültig
Kind, Erwachsenenbereich	falsch markiert	Bereich passt nicht

Fünf Fälle, fünf Fehler — bei einer Funktion, die man in jeder Durchsicht durchwinken würde. Die dritte Zeile ist die schwerwiegendste: Was der Arzt braucht, wird nicht markiert; was er nicht braucht, wird markiert.

Der Alarmermüdung auf der Spur

Regel	Markiert	davon relevant	Quote
Außerhalb Bereich	**184**/Tag	9	**5** %
+ stabile ausgenommen	61	9	15 %
+ Veränderung	**23**	**14**	**61** %

Die erste Zeile ist der Zustand, den Band 18 als Alarmermüdung beschreibt: Fünf Prozent Trefferquote bedeutet, dass neunzehn von zwanzig Markierungen falsch sind — und nach zwei Wochen sieht niemand mehr hin. Die dritte Zeile markiert weniger und findet mehr.

Bemerkenswert ist die mittlere Spalte: Von neun auf vierzehn relevante Fälle. Die dritte Regel findet Dinge, die die erste übersieht *— die Verschlechterung innerhalb des Normbereichs.*

Aufgabe 4.1.

(a) Entwickeln Sie die Auffälligkeitsregel neu.

(b) Bestimmen Sie, wo die Fallunterscheidung hingehört.

(c) Untersuchen Sie, wann Kürze schadet.

Lösung Schritt für Schritt.

(a) Die Regel neu entwickeln.
Schritt 1: Die Frage klären. Was soll die Markierung auslösen?

Anlass	Relevant?	
Neu außerhalb	**ja**	Zustandswechsel
Weiterhin außerhalb, stabil	nein	**bekannt**
Deutliche Änderung	**ja**	auch im Bereich
Kritischer Wert	**immer**	unabhängig vom Verlauf
Ungültige Probe	ja	**andere Art**

Fünf Anlässe, und sie sind nicht gleichartig: Die letzte Zeile ist keine Auffälligkeit des Wertes, sondern eine der Probe. Wer sie in dieselbe Funktion zwingt, vermischt zwei Fragen — ein Wert kann nicht zugleich zu hoch und ungültig sein.

Schritt 2: Der zweite Entwurf.

```
class Anlass(Enum):
NEU_AUSSERHALB = auto()
KRITISCH = auto()
AENDERUNG = auto()
UNGUELTIG = auto()

def anlaesse(b, vorher):
"""vorher:  letzter gueltiger Befund oder None."""
if b.qualitaet & UNBRAUCHBAR:
return {Anlass.UNGUELTIG}
if not isinstance(b.wert, Zahlwert):
return set()
gefunden = set()
if b.bereich.kritisch(b.wert):
gefunden.add(Anlass.KRITISCH)
war_aussen = vorher and b.bereich.aussen(vorher.wert)
if b.bereich.aussen(b.wert) and not war_aussen:
gefunden.add(Anlass.NEU_AUSSERHALB)
if vorher and b.groesse.deutlich(vorher.wert, b.wert):
gefunden.add(Anlass.AENDERUNG)
return gefunden
```

Aus einem Wahrheitswert wird eine Menge von Anlässen. Das ist der eigentliche Entwurfsschritt: Die Frage „ist es auffällig“ hat keine Ja-Nein-Antwort, sondern eine Begründung — und die Oberfläche kann verschiedene Anlässe verschieden darstellen.

Zwei Einzelheiten. Die Ungültigkeit verlässt die Funktion sofort, weil ein ungültiger Wert nicht bewertet werden darf. Und `deutlich` *sitzt an der Messgröße, nicht in der Funktion — weil zehn Prozent Änderung bei Natrium alarmierend und bei Ferritin belanglos sind.*

(b) Wo die Fallunterscheidung hingehört.

Ort	Vorteil	Nachteil
In der Funktion	alles an einer Stelle	**wächst mit jeder Größe**
Am Datentyp	**je Größe eigen**	mehr Typen
In einer Tabelle	ohne Code änderbar	keine Prüfung

Die dritte Zeile ist verlockend und gefährlich: Fachliche Regeln in einer Tabelle lassen sich ohne Programmierer ändern — und niemand prüft, ob die Änderung sinnvoll ist. Sie ist richtig für Grenzwerte, die sich ändern, und falsch für Regeln, die Struktur haben.

Die Aufteilung im Fall.

Was	Wohin	Warum
Referenz-grenzen	**Tabelle**	ändern sich je Labor
Kritische Grenzen	Tabelle	
Schwelle „deutlich“	**Messgröße**	je Größe verschieden
Ablauf der Prüfung	**Code**	hat Struktur

Die Trennlinie verläuft zwischen Werten, *die sich ändern, und* Regeln, *die Struktur haben. Wer Struktur in Tabellen legt, baut eine Programmiersprache in Daten nach — und die hat dann keinen Übersetzer, keine Prüfung und keine Fehlermeldungen.*

(c) Wann Kürze schadet.
Der Fall. Dieselbe Auswertung, zwei Fassungen.

```
# Fassung A
r = [x for s in d.values() for x in s if x.t > g
and
x.c in k and not any(y.e == x.id for y in a)]
```

```
# Fassung B
ersetzte = {b.ersetzt for b in alle if b.ersetzt}
aktuell = [b for b in alle if b.id not in ersetzte]
im_zeitraum = [b for b in aktuell if b.abnahme >
grenze]
gesucht = [b for b in im_zeitraum
if b.groesse.code in codes]
```

Maß	Fassung A	Fassung B	
Zeilen	**3**	6	
Zeichen	**118**	246	
Verstehenszeit	**95 s**	**22 s**	Faktor 4,3
Fehler beim Ändern	3/8	**0/8**	
Zwischen-schritt prüfbar	nein	**ja**	

Fassung A ist halb so lang und braucht viermal so lange zum Verstehen. Der Grund steht in Band 18, Kapitel 2: Jeder unbenannte Zwischenschritt muss im Arbeitsgedächtnis gehalten werden, und Fassung A verlangt vier gleichzeitig.

Die Regel.

Nicht	Möglichst wenige Zeilen
Sondern	**Möglichst wenige Dinge gleichzeitig im Kopf**
Das Mittel	Zwischenergebnisse benennen — ein Name ist eine Erklärung

Ein benannter Zwischenschritt kostet eine Zeile und eine Variable, und er spart dem Leser eine Einheit im Gedächtnis. Bei einer Kapazität von vier ist das ein Viertel — der beste Tauschhandel, den es beim Schreiben gibt.

Kürze ist nicht Klarheit

Eigenschaft	Messbar?	
Zeilenzahl	ja	**sagt wenig**
Zeichenzahl	ja	sagt wenig
Verstehenszeit	**ja**	**sagt alles**

Die dritte Zeile lässt sich mit zwei Kollegen und einer Stoppuhr erheben: Man stellt eine Frage zum Code und misst, wie lange die Antwort dauert. Kapitel 8 führt das Verfahren aus — es ist die einzige Lesbarkeitsmessung, die etwas taugt.

Damit ist Lesbarkeit keine Geschmacksfrage mehr. Wer behauptet, seine dichte Fassung sei klarer, kann das prüfen lassen — und die Prüfung fällt bemerkenswert oft gegen ihn aus.

Bentley[4] führt vor, wie aus einer naheliegenden Lösung eine gute wird; Kernighan und Pike[11] behandeln Einfachheit als Handwerksregel; McConnell[16] sammelt die empirischen Befunde zu Verständlichkeit und Fehlerrate.

5 Zerlegung

Warum man zerlegt

Grund	Trägt?	
„Funktionen sollen kurz sein“	**nein**	Zahl ohne Begründung
Wiederverwendung	selten	**meist erfunden**
Prüfbarkeit	ja	
Man kann einen Teil verstehen, ohne das Ganze zu kennen	**ja**	**der eigentliche Grund**

Die letzte Zeile ist der Maßstab. Eine gute Zerlegung erlaubt es, eine Funktion zu lesen und zu beurteilen, ohne ihre Aufrufer und ihre Aufgerufenen zu kennen. Wo das nicht geht, hat man den Code zerteilt, aber nicht zerlegt.

Der Unterschied.

	Zerteilen	Zerlegen
Schnitt entlang	**Zeilenzahl**	**Zuständigkeit**
Teil verständlich	nur im Zusammenhang	für sich
Name	`teil2`, `verarbeite`	**sagt, was gilt**
Argumente	viele	wenige

Die Zahl der Argumente ist der beste Schnelltest. Eine Funktion mit acht Argumenten hat keine eigene Zuständigkeit — sie ist ein herausgeschnittenes Stück, das seinen ganzen Kontext mitschleppen muss.

Ousterhouts Maßstab: tiefe Schnittstellen

	Flach	Tief
Schnittstelle	**breit**	**schmal**
Dahinter	wenig	viel
Nutzen	gering	hoch
Beispiel	`setze_x(w)`	`verlauf(pat, groesse, raum)`

Der Nutzen einer Funktion ist die Menge an Komplexität, die sie verbirgt, geteilt durch die Menge, die man kennen muss, um sie zu benutzen. Eine Funktion, die genauso viel offenlegt wie sie verbirgt, kostet mehr, als sie bringt.

Die Probe an unserem Fall.

Funktion	Argum.	Bewertung
`verlauf(pat, code, raum)`	3	**tief**
`anlaesse(befund, vorher)`	2	tief
`formatiere(w, e, n, d, s, f)`	6	**flach**
`pruefe(b, r, k, q, v, m, t)`	7	zerteilt

Die dritte Zeile ist der typische Fall: Eine Formatierfunktion, die Wert, Einheit, Nachkommastellen, Dezimaltrenner, Sprache und Feldbreite entgegennimmt. Sie verbirgt fast nichts — der Aufrufer muss all das ohnehin wissen und beschaffen.

Wenn viele Argumente auftreten

Der Befund	Sechs Argumente, die immer zusammen auftreten
Die Deutung	**Es fehlt ein Begriff**
Die Abhilfe	Ein Typ, der sie zusammenfasst — und meist einen Namen hat, den die Fachsprache schon kennt

Wert, Einheit, Nachkommastellen und Trenner sind zusammen eine Größenangabe. *Der Begriff existiert in der Fachsprache; er fehlte nur im Programm. Wo Argumente in Gruppen wandern, ist fast immer ein Begriff übersehen worden — und das ist der häufigste Weg, ein Datenmodell zu verbessern.*

Aufgabe 5.1.

(a) Zerlegen Sie eine gewachsene Funktion.
(b) Bestimmen Sie die richtige Schnittgröße.
(c) Untersuchen Sie, wann Zerlegung schadet.

Lösung Schritt für Schritt.

(a) Eine gewachsene Funktion zerlegen.
Der Ausgangszustand. Eine Funktion `zeige_verlauf`, 140 Zeilen, über vier Jahre gewachsen.

Was sie tut	Zeilen	Zuständigkeit
Befunde holen	22	**Beschaffung**
Ersetzte aussortieren	14	Fachlogik
Anlässe bestimmen	31	**Fachlogik**
Werte umrechnen	18	Fachlogik
Achsen berechnen	26	**Darstellung**
Ausgeben	29	Darstellung

Die rechte Spalte macht den Schnitt sichtbar: drei verschiedene Zuständigkeiten. Der naheliegende Fehler wäre, nach Zeilenzahl in sechs Funktionen zu je zwanzig Zeilen zu schneiden — dann hätte man sechs Teile, die einander alle kennen müssen.

Der Schnitt entlang der Zuständigkeit.

Teil	Zeilen	Argumente	Prüfbar?
Beschaffung	22	3	braucht Datenbank
Fachlogik	63	2	**rein — trivial**
Darstellung	55	2	gegen Ausgabe

Die mittlere Zeile ist der Gewinn: Dreiundsechzig Zeilen Fachlogik ohne Datenbank, ohne Ausgabe, ohne Uhrzeit — prüfbar mit einer Liste von Befunden und einer erwarteten Ausgabe. Das ist die wertvollste Eigenschaft dieser Zerlegung und kein Nebeneffekt, sondern der Zweck.

Die Wirkung auf die Prüfbarkeit.

	Vorher	Nachher	
Prüfbare Zeilen	0	**63**	45 %
Aufbau je Prüfung	Datenbank	**eine Liste**	
Dauer je Prüfung	≈ 800 ms	< **1 ms**	
Prüffälle geschrieben	2	**31**	

Die letzte Zeile ist die eigentliche Wirkung: Nicht die Möglichkeit zu prüfen entscheidet, sondern die Bequemlichkeit. Bei achthundert Millisekunden Aufbau schreibt niemand einunddreißig Fälle; bei einer Millisekunde schreibt man sie nebenbei.

(b) Die richtige Schnittgröße.

Zeilen	Innere Komplexität	Schnittstellen	Summe
3–5	gering	**sehr viele**	hoch
15–40	gering	wenige	**gering**
80–150	spürbar	sehr wenige	mittel
> 300	hoch	keine	hoch

Es gibt ein Optimum, kein Maximum — dieselbe Kurve wie bei den Modulen in Band 17, Kapitel 4. Wer in Dreizeiler zerlegt, hat die Komplexität nicht beseitigt, sondern in die Aufrufkette verschoben, wo sie schwerer zu überblicken ist.

Der bessere Maßstab als Zeilenzahl.

Nicht	„Passt sie auf den Bildschirm?“
Sondern	**„Kann ich in einem Satz sagen, was sie garantiert?“**
Und	Ohne „und“ in diesem Satz

Das „und“ ist der Prüfstein. „Holt die Befunde und *bestimmt die Anlässe“ sind zwei Zuständigkeiten. „Liefert die gültigen Befunde einer Größe im Zeitraum, aufsteigend“ ist eine — gleichgültig, ob sie zehn oder achtzig Zeilen braucht.*

(c) Wann Zerlegung schadet.

Fall	Zerlegen?	
Ein zusammenhängendes Verfahren	**nein**	**Schritte hängen ab**
Nur an einer Stelle benutzt, trivial	nein	Namensfindung kostet mehr
Zwischenschritt ohne Bedeutung	nein	**kein Begriff dafür**
Wiederkehrende Zuständigkeit	ja	

Die dritte Zeile ist das beste Erkennungszeichen: Wenn einem für einen herausgelösten Teil kein Name einfällt, der mehr sagt als `schritt2`, dann ist es keine eigene Zuständigkeit. Der fehlende Name ist die Warnung — man sollte sie ernst nehmen statt einen zu erfinden.

Ein Gegenbeispiel.

```
# schlecht zerlegt
def hole_liste(p):  ...
def filtere_liste(l):  ...
def sortiere_liste(l):  ...
def formatiere_liste(l):  ...
```

Prüffrage	Antwort	
Was garantiert `filtere_liste`?	„filtert“	**nichts gesagt**
Kann man sie einzeln verstehen?	nein	**welche Liste, welcher Filter?**
Wird sie anderswo benutzt?	nein	

Vier Funktionen mit vier nichtssagenden Namen — die Zerlegung folgt den Verarbeitungsschritten statt den Zuständigkeiten. Das ist genau Parnas' schlechte Zerlegung aus Band 17, Kapitel 4, hier auf Funktionsebene.

Der Name ist die Zerlegung

Die Reihenfolge	Erst den Namen finden, dann die Funktion schreiben
Der Grund	**Wer den Namen nicht findet, hat die Zuständigkeit nicht**
Der Nebeneffekt	Der Name deckt fehlende Begriffe im Datenmodell auf

Das klingt nach Formalismus und ist ein praktisches Werkzeug. Der Versuch, eine Funktion zu benennen, bevor man sie schreibt, scheitert genau dann, wenn sie keine klare Aufgabe hat — und dieses Scheitern ist wertvoller als jede spätere Durchsicht.

Umgekehrt gilt: Ein Name, der die Fachsprache benutzt — `gueltige_befunde`, `anlaesse`, `medikation_am` — ist meist ein Zeichen, dass die Zerlegung entlang der Sache verläuft und nicht entlang der Technik.

Parnas[20] begründete das Kriterium der Zerlegung; Ousterhout[19] formuliert es als Forderung nach tiefen Schnittstellen neu; Beck[3] sammelt die kleinen Entscheidungen, aus denen sich Lesbarkeit zusammensetzt.

6 Zustand

Warum Zustand die Schwierigkeit ist

	Ohne Zustand	Mit Zustand
Verstehen	**die Funktion allein**	**ihre ganze Geschichte**
Prüfen	Eingabe, Ausgabe	Vorzustand herstellen
Fehlersuche	reproduzierbar	**von der Folge abhängig**
Nebenläufig	unkritisch	Wettlauf möglich

Die erste Zeile ist der Kern. Eine Funktion ohne Zustand versteht man aus ihrem Text; bei einer mit Zustand muss man wissen, was vorher geschah. Das ist genau die Belastung, die Band 18 als darstellungsbedingt bezeichnet — sie folgt nicht aus der Aufgabe, sondern aus der Bauweise.

Die Zustandsarten, nach Schwierigkeit geordnet.

Art	Reichweite	Bewertung
Lokale Variable	**eine Funktion**	harmlos
Feld eines Objekts	ein Objekt	beherrschbar
Modulvariable	**das Programm**	schwierig
Globaler Zustand	alles	**sehr schwierig**
Zustand in der Datenbank	**über Läufe hinweg**	**am schwierigsten**

Die letzte Zeile wird selten mitgezählt und ist die härteste: Ein Fehler, der von einem Datenbankzustand abhängt, tritt nur bei bestimmten Beständen auf — und der Bestand ist nach dem nächsten Lauf ein anderer.

Der Fall: die Warnungsliste

Der erste Entwurf. Ein Objekt hält die aktuellen Warnungen eines Patienten.

```
class Warnungsliste:
def __init__(self):
self.warnungen = []

def pruefe(self, befund, vorher):
for a in anlaesse(befund, vorher):
self.warnungen.append(Warnung(befund, a))

def bestaetige(self, index):
del self.warnungen[index]
```

Zehn Zeilen, verständlich, und sie enthalten vier Fehler — von denen drei erst auftreten, wenn zwei Dinge gleichzeitig geschehen. Genau solche Fehler sind es, die in einem KIS Wochen zur Aufklärung brauchen.

Die vier Fehler.

Fehler	Auslöser	Folge
Index nach Änderung	**Liste hat sich verschoben**	**falsche Warnung bestätigt**
Doppelte Warnung	Befund zweimal geprüft	Rauschen
Keine Historie	bestätigt = gelöscht	**nicht nachvollziehbar**
Wettlauf	**zwei Benutzer**	Liste inkonsistent

Die erste Zeile ist der klassische Fall und in der Klinik gefährlich: Zwischen Anzeige und Bestätigung kommt ein neuer Befund herein, die Liste verschiebt sich, und der Arzt bestätigt eine Warnung, die er nie gesehen hat.

Kennung statt Position

	Über Position	Über Kennung
Stabil bei Änderung	**nein**	**ja**
Fehlerbild	falsches Element	nicht gefunden
Bemerkt	nie	sofort

Die letzten beiden Zeilen sind der eigentliche Gewinn. Ein Zugriff über die Position schlägt lautlos *fehl — er trifft das falsche Element. Ein Zugriff über eine Kennung schlägt hörbar fehl — das Element gibt es nicht. Ein lauter Fehler ist immer besser als ein leiser.*

Aufgabe 6.1.

(a) Entwerfen Sie die Warnungsliste ohne veränderlichen Zustand.
(b) Verfolgen Sie einen Wettlauf.
(c) Bestimmen Sie, wo Zustand unvermeidlich ist.

Lösung Schritt für Schritt.

(a) Ohne veränderlichen Zustand.
Der Gedanke. Statt eine Liste zu führen, wird sie bei Bedarf berechnet — wie die Medikation in Kapitel 2.

```
@dataclass(frozen=True)
class Bestaetigung:
warnung:  WarnungId
von:  BenutzerId
am:  datetime

def offene_warnungen(befunde, bestaetigungen,
zeitpunkt):
erledigt = {b.warnung for b in bestaetigungen}
return [w for w in alle_warnungen(befunde,
zeitpunkt)
if w.id not in erledigt]
```

Der Zustand ist nicht verschwunden — er ist von einer veränderlichen Liste zu einer wachsenden Menge von Ereignissen geworden. Nichts wird gelöscht, nichts überschrieben; es kommt nur hinzu.

Was die vier Fehler machen.

Fehler	Noch möglich?	
Index nach Änderung	**nein**	**Kennung statt Position**
Doppelte Warnung	nein	**Kennung aus Befund und Anlass**
Keine Historie	**nein**	Bestätigungen bleiben
Wettlauf	nein	**nur Anfügen**

Vier Fehler, alle vier verschwunden — und zwar nicht durch Sorgfalt, sondern weil die Bauform sie nicht mehr zulässt. Das ist derselbe Unterschied wie in Band 19: einen Fehler unmöglich machen statt ihn zu vermeiden.

Der Preis.

	Liste	Ereignisse	
Zeilen	10	**14**	
Speicher	$\mathcal{O}(\text{offen})$	$\mathcal{O}(\text{alle})$	wächst
Lesen	$\mathcal{O}(1)$	$\mathcal{O}(n)$	oder Zwischen-speicher
Fehlerklassen	**4**	**0**	**der Gewinn**

Vier Zeilen mehr, unbegrenzt wachsender Speicher und langsameres Lesen — gegen vier verschwundene Fehlerklassen. Bei einem klinischen System, in dem die Nachvollziehbarkeit ohnehin vorgeschrieben ist, ist das kein Handel, sondern die einzige richtige Bauform.

(b) Einen Wettlauf verfolgen.
Die Lage. Zwei Ärzte bestätigen gleichzeitig Warnungen desselben Patienten.

Zeit	Sitzung A	Sitzung B
t_1	liest Liste $[w_1, w_2, w_3]$	
t_2		liest Liste $[w_1, w_2, w_3]$
t_3	`bestaetige(0)` → w_1 weg	
t_4		`bestaetige(1)`
t_5		**löscht w_3 statt w_2**

Sitzung B wollte w_2 bestätigen und hat w_3 getroffen, weil sich die Liste zwischen Lesen und Schreiben verschoben hat. Niemand bemerkt es — w_2 erscheint weiterhin, w_3 ist stillschweigend erledigt.

Warum das schwer zu finden ist.

Eigenschaft	Wert	
Tritt auf	**selten**	bei Gleichzeitigkeit
Reproduzierbar	nein	**zeitabhängig**
Fehlermeldung	keine	lautloser Fehler
Symptom	**„Warnung war plötzlich weg“**	klingt nach Bedienfehler

Die letzte Zeile ist der Grund, warum solche Fehler jahrelang überleben: Die Meldung klingt wie ein Bedienfehler, lässt sich nicht nachstellen, und die Untersuchung endet mit „nicht reproduzierbar“. Kapitel 7 behandelt, wie man dennoch weiterkommt.

(c) Wo Zustand unvermeidlich ist.

Bereich	Zustand nötig	
Fachlogik	**nein**	**rein rechenbar**
Datenhaltung	ja	das ist ihr Zweck
Sitzung	ja	begrenzt
Oberfläche	ja	kurzlebig

Zustand lässt sich nicht abschaffen — ein Programm ohne Zustand tut nichts Bleibendes. Er lässt sich aber konzentrieren*: Wenn die Fachlogik rein ist und aller Zustand an den Rändern sitzt, ist der schwierige Teil klein und der große Teil einfach.*

Die Aufteilung im Fall.

Schicht	Zeilen	Zustand	Prüfbar
Oberfläche	2100	ja	aufwendig
Anwendungs-ablauf	600	Übergabe	mittel
Fachlogik	**1800**	**nein**	**trivial**
Datenzugriff	900	ja	gegen Testbestand

Ein Drittel des Codes ist reine Fachlogik und damit ohne Aufbau prüfbar — und genau dort sitzen die Regeln, deren Richtigkeit klinisch zählt. Das ist die Aufteilung, auf die dieses Kapitel hinausläuft.

Zustand versteckt sich

Erscheinung	Zustand?	
Zwischenspeicher	**ja**	**fällt niemandem auf**
Verbindungspool	ja	
Zufallszahlen-generator	ja	Folge hängt ab
Protokollierung	ja	
`now()`	**ja**	**Kapitel 3**

Ein Zwischenspeicher macht aus einer reinen Funktion eine mit Gedächtnis — und aus einem reproduzierbaren Fehler einen, der beim zweiten Aufruf verschwindet. Er ist die häufigste Ursache für Fehler, die sich nicht nachstellen lassen.

Deshalb lautet die Regel: Zwischenspeicher gehören an den Rand, nicht in die Fachlogik — und sie müssen abschaltbar sein. Ein Fehler, der bei abgeschaltetem Zwischenspeicher verschwindet, ist damit schon zur Hälfte gefunden.

Ramalho[22] zeigt die Sprachmittel für unveränderliche Werte in Python; Percival und Gregory[21] führen die Trennung von reiner Fachlogik und zustandsbehafteten Rändern durch; Fowler[7] beschreibt die Schritte, mit denen man Zustand nachträglich herausdrängt.

Part III

Das Handwerk

7 Fehlersuche als Verfahren

Fehlersuche ist keine Begabung

	Was viele tun	Was ein Verfahren ist
Vorgehen	**Code anstarren**	**Hypothesen prüfen**
Auswahl	Bauchgefühl	Halbierung
Ende	„jetzt geht's“	**Ursache benannt**
Wiederholbar	nein	ja
Lehrbar	nein	**ja**

Die dritte Zeile trennt die Handwerker von den Bastlern. Wer aufhört, sobald das Symptom verschwindet, hat nicht den Fehler behoben, sondern eine Bedingung geändert, unter der er sich zeigte. Er kommt wieder, an anderer Stelle.

Das Verfahren in sieben Schritten.

1	**Reproduzieren** — verlässlich, mit möglichst wenigen Schritten
2	**Verkleinern** — den Fall auf das Minimum reduzieren
3	Beobachten, was tatsächlich geschieht — nicht was geschehen sollte
4	**Eine Hypothese bilden**, die das Beobachtete erklärt
5	Eine Vorhersage ableiten, die sie *widerlegen* könnte
6	Prüfen. Bei Widerspruch: neue Hypothese
7	**Ursache benannt** — erst dann beheben

Schritt 5 ist der, den fast niemand macht: eine Vorhersage suchen, die die eigene Hypothese widerlegen *würde. Wer nur bestätigende Beobachtungen sucht, findet sie immer — und behebt am Ende das Falsche.*

Warum Schritt 1 der wichtigste ist

Reproduzier-barkeit	Zeit bis Fund	
Immer	≈ 1 h	Halbierung möglich
Meistens	≈ 4 h	
Gelegentlich	≈ 3 Tage	
Nie	∞	**Raten**

Die Zeit bis zum Fund hängt fast vollständig an der Reproduzierbarkeit. Deshalb ist es fast immer richtig, Stunden in einen verlässlichen Auslöser zu investieren, bevor man eine einzige Zeile ansieht — und fast immer falsch, sofort im Code zu suchen.

Der Fall: „Die Warnung war plötzlich weg“

Die Meldung. Eine Stationsärztin berichtet, eine Warnung sei verschwunden, ohne dass sie sie bestätigt habe. Zweimal in drei Wochen.

Erste Reaktion	„Nicht reproduzierbar, vermutlich Bedienfehler“
Warum falsch	**Die Meldung enthält bereits eine Information**
Welche	Es *gibt* einen Weg, eine Warnung zu entfernen, ohne sie zu bestätigen

Die dritte Zeile ist der Einstieg. Statt zu fragen „hat sie sich geirrt“, fragt man: Welche Wege gibt es überhaupt, an denen eine Warnung verschwindet? Das ist eine Frage an den Code und in zwanzig Minuten beantwortbar.

Schritt 2: Die Wege aufzählen.

Weg	Prüfbar?	Passt zur Meldung?
Bestätigung	Protokoll	keine verzeichnet
Befund ersetzt	Historie	möglich
Bereich geändert	Stammdaten	nein
Wettlauf	**nicht protokolliert**	**möglich**

Vier Wege, zwei davon möglich — damit ist aus „nicht reproduzierbar“ eine Liste mit zwei Einträgen geworden. Das ist der ganze Trick von Schritt 2: Man verkleinert nicht den Testfall, sondern den Raum der Erklärungen.

Schritt 5: Die widerlegende Vorhersage.

Hypothese	Der Wettlauf aus Kapitel 6
Wenn sie stimmt	Dann muss eine *andere* Warnung bestätigt worden sein als angezeigt
Die Vorhersage	**Im Protokoll steht eine Bestätigung für eine Warnung, die niemand gesehen haben kann**

Das ist eine Vorhersage, die falsch sein kann — und genau deshalb taugt sie. Eine Abfrage über das Protokoll beantwortet sie in Minuten. Sie traf zu: zwei Bestätigungen für Warnungen, die zum Zeitpunkt der Anzeige noch nicht existierten.

Aufgabe 7.1.

(a) Halbieren Sie systematisch.
(b) Verkleinern Sie einen Fehlerfall.
(c) Bestimmen Sie, wann Ausgabe besser ist als ein Fehlersuchprogramm.

Lösung Schritt für Schritt.

(a) Systematisch halbieren.
Die Lage. Seit einer von 180 Übergaben ist ein Verlauf falsch sortiert.

Schritt	Bereich	Geprüft	Ergebnis
1	1–180	90	gut
2	91–180	135	schlecht
3	91–134	112	gut
4	113–134	123	schlecht
5	113–122	117	gut
6–8			→ **Übergabe** 121

Acht Prüfungen für hundertachtzig Übergaben — $\log_2 180 = 7{,}5$. *Bei zehn Minuten je Prüfung sind das gut eine Stunde, und das Ergebnis ist eine einzelne Übergabe mit meist wenigen geänderten Zeilen.*

Die Voraussetzungen.

Bedingung	Nötig?	
Verlässlicher Prüfschritt	**ja**	**Schritt 1**
Jede Übergabe lauffähig	ja	Band 17, Kap. 10
Kleine Übergaben	sehr hilfreich	Ergebnis genauer

Die letzte Zeile ist der versteckte Gewinn kleiner Übergaben: Sie machen die Halbierung nicht schneller, aber ihr Ergebnis schärfer. Bei einer Übergabe mit dreißig geänderten Dateien ist man am Ende so klug wie vorher.

(b) Einen Fehlerfall verkleinern.
Der Ausgangsfall. Ein Absturz beim Laden eines Patienten mit 3400 Befunden.

Schritt	Befunde	Absturz?	Dauer
Ausgangslage	3400	ja	40 s
Erste Hälfte	1700	nein	
Zweite Hälfte	1700	ja	
Weiter halbieren	212	ja	
⋮	⋮		
Minimum	**2**	**ja**	**0,1 s**

Aus dreitausendvierhundert Befunden werden zwei — und aus vierzig Sekunden Wartezeit ein Zehntel. Der verkleinerte Fall ist nicht nur schneller, er ist lesbar*: Zwei Befunde kann man ansehen und verstehen, warum sie das Problem auslösen.*

Was das Minimum verriet.

Befund A	Ein Zahlwert, Kalium, validiert
Befund B	**Ein Grenzwert `< 0.1` derselben Größe**
Die Ursache	Der Sortiervergleich griff auf `.betrag` zu — den ein `Grenzwert` nicht hat

Genau der Fall aus Kapitel 2, den der Typentwurf sichtbar gemacht hatte. Das Verkleinern hat ihn in zwölf Minuten freigelegt — die Durchsicht von dreitausendvierhundert Befunden hätte ihn nie gefunden.

Der Gewinn im Vergleich.

Vorgehen	Dauer	
Code durchlesen	> 4 h	ohne Erfolg
Ablaufverfolgung ansehen	$\approx$ 90 min	Stapel 60 Ebenen tief
Fall verkleinern	**12 min**	**Ursache offen**

Das Verkleinern ist mechanisch, verlangt kein Verständnis des Codes und lässt sich sogar automatisieren. Es ist das wirksamste einzelne Werkzeug der Fehlersuche — und wird am seltensten benutzt, weil es unintellektuell aussieht.

(c) Ausgabe gegen Fehlersuchprogramm.

	Schrittweise	**Ausgabe**
Gut für	**ein Ablauf, tief**	**viele Läufe, Muster**
Nebenläufigkeit	**verändert das Verhalten**	brauchbar
Seltener Fehler	man müsste warten	**läuft mit**
Beim Kunden	nicht möglich	ja
Zeitverhalten	verfälscht	beeinflusst

Die zweite Zeile ist der entscheidende Punkt bei nebenläufigen Fehlern: Ein Haltepunkt hält einen Faden an und verändert damit genau das Zeitverhalten, das den Fehler auslöst. Der Fehler verschwindet, sobald man hinsieht — und das ist kein Zufall, sondern eine Folge der Methode.

Was eine gute Ausgabe ausmacht.

Nicht	`print("hier"),print(x)`
Sondern	**Zeitpunkt, Faden, Bezeichner, Wert — maschinell auswertbar**
Der Grund	Bei einem seltenen Fehler hat man *einen* Mitschnitt und muss ihn auswerten können

Eine strukturierte Ausgabezeile lässt sich filtern, zählen und sortieren; ein `print` nicht. Der Unterschied kostet beim Schreiben zehn Sekunden und entscheidet, ob man den einen Mitschnitt auswerten kann, den man bekommt.

Der Fehler ist selten dort, wo er auffällt

Symptom	Auffallende Stelle	Tatsächliche Ursache
Absturz beim Sortieren	Vergleichs-funktion	**Typentwurf**
Falscher Verlauf	Anzeige	Zeitpunktwahl
Warnung verschwindet	Bestätigung	**Zugriff über Position**
Langsam	Datenbank	$\mathcal{O}(n^2)$ im Code

In allen vier Fällen liegt die Ursache eine oder zwei Ebenen vor dem Symptom. Wer an der auffallenden Stelle repariert, behebt die Erscheinung — und der Fehler kehrt an anderer Stelle zurück. Das ist der Unterschied zwischen Schritt 6 und Schritt 7 des Verfahrens.

Die Prüffrage lautet: Kann ich erklären, warum der Fehler genau jetzt *und genau* hier *auftrat — und warum er vorher nicht auftrat? Wer das nicht kann, hat die Ursache nicht.*

Zeller[28] macht die Fehlersuche zum systematischen Verfahren mit Hypothesen und Verkleinerung; Kernighan und Pike[11] behandeln sie als tägliches Handwerk; Bentley[5] zeigt an Fällen, wie weit man mit Beobachtung statt Vermutung kommt.

8 Fremden Code lesen

Die unterschätzte Tätigkeit

	Anteil der Zeit	Wird geübt
Schreiben	≈ 20 %	ständig
Lesen	≈ **30** %	**nie**

> *Man liest anderthalbmal so viel, wie man schreibt, und hat es nie gelernt. In keiner Ausbildung wird Codelesen geübt, in keinem Buch als Fertigkeit behandelt — obwohl es genauso ein Handwerk ist wie die Fehlersuche und dieselbe Systematik verträgt.*

Warum es schwer ist.

	Prosa	Code
Reihenfolge	**linear**	**Sprünge, Aufrufe**
Zusammenhang	im Text	oft in anderen Dateien
Gedächtnislast	gering	**mehrere Ebenen zugleich**
Zweck	steht da	**steht nirgends**

Die letzte Zeile ist die eigentliche Schwierigkeit. Der Code sagt, was *geschieht — er sagt nie, warum es so und nicht anders geschieht. Genau diese Frage muss der Leser rekonstruieren, und dafür gibt es Verfahren.*

Fünf Zugänge, nach Aufwand geordnet

Zugang	Beantwortet	Aufwand
Über die Daten	**Was gibt es?**	gering
Über die Ränder	Was kommt herein?	gering
Über die Historie	**Warum so?**	gering
Über einen Ablauf	Was geschieht?	mittel
Von vorn lesen	—	**hoch, nutzlos**

Die letzte Zeile ist der Zugang, den fast alle wählen und der am schlechtesten funktioniert. Code hat keinen Anfang — er ist ein Graph, kein Text. Wer bei Zeile eins beginnt, liest in beliebiger Reihenfolge und behält nichts.

Der erste Zugang: über die Daten.

1	Alle Typen und Datenklassen auflisten — meist wenige Dutzend
2	Nach Zusammengehörigkeit gruppieren
3	**Die Beziehungen zwischen ihnen skizzieren**
4	Erst dann Funktionen ansehen

Wirths Rangordnung aus Kapitel 2, für das Lesen umgedreht: Wer die Datenstrukturen kennt, errät die Verfahren. Bei einem System mit 40 000 *Zeilen sind das etwa dreißig Typen — ein Nachmittag, und danach versteht man Funktionen beim ersten Lesen.*

Der dritte Zugang: die Historie befragen

Die Frage	Warum steht hier diese seltsame Bedingung?
Das Werkzeug	Zeilenweise Zuordnung zur Übergabe
Was man findet	**Die Übergabemeldung, oft mit Fallnummer**

Eine Bedingung, die keinen Sinn ergibt, ist fast immer die Antwort auf einen Fehlerbericht. Die Historie sagt, auf welchen — und damit versteht man in zwei Minuten, was aus dem Code allein nicht hervorgeht. Das ist der wirksamste Lesezugang überhaupt und wird am seltensten benutzt.

Umgekehrt folgt daraus eine Schreibregel: Die Übergabemeldung soll das Warum *enthalten, nicht das Was. Was geändert wurde, steht im Unterschied; warum, steht nirgends sonst.*

Aufgabe 8.1.

(a) Erschließen Sie ein unbekanntes System.
(b) Messen Sie Lesbarkeit.
(c) Untersuchen Sie, was das Lesen erschwert.

Lösung Schritt für Schritt.

(a) Ein unbekanntes System erschließen.
Die Lage. 38 000 Zeilen, keine Dokumentation, niemand mehr im Haus, der es geschrieben hat.

Std.	Tätigkeit	Ergebnis	Verständnis
0–1	Verzeichnisse, Dateigrößen	Grobbild	5 %
1–3	**Alle Typen auflisten**	**34** Typen	**25** %
3–4	Beziehungen skizzieren	eine Seite	40 %
4–5	Ränder: Eingänge, Datenbank	12 Eingänge	50 %
5–7	**Einen Ablauf verfolgen**	Aufrufkette	**65** %
7–8	Historie der auffälligen Stellen	9 Fallnummern	**75** %

Acht Stunden für fünfundsiebzig Prozent — und keine einzige davon mit Lesen von vorn. Die zweite Zeile trägt den größten Sprung: Vierunddreißig Typen aufzulisten dauert zwei Stunden und verfünffacht das Verständnis.

Zum Vergleich: der übliche Weg.

Vorgehen	Nach 8 h	Nach 40 h	
Von vorn lesen	≈ 15 %	≈ 55 %	unstrukturiert
An der Aufgabe entlang	≈ 35 %	≈ 70 %	lückenhaft
Systematisch	**75** %	≈ **90** %	tragfähig

Die mittlere Zeile ist der Normalfall: Man bekommt eine Aufgabe und liest nur, was man dafür braucht. Das geht schnell und erzeugt ein Verständnis mit Löchern — und die Löcher merkt man erst, wenn man etwas ändert, das man nicht überblickt hat.

(b) Lesbarkeit messen.
Das Verfahren. Man stellt Fragen und misst die Antwortzeit.

1	Fünf Fragen zum Code formulieren, die eine sachliche Antwort haben
2	Drei Personen, die den Code nicht kennen
3	**Zeit bis zur richtigen Antwort messen**
4	Falsche Antworten getrennt zählen

Frage	Fassung A	Fassung B	Faktor
Was gibt sie zurück?	38 s	**9** s	4,2
Was bei leerer Liste?	71 s	**14** s	5,1
Wo wird sortiert?	22 s	6 s	3,7
Was, wenn zwei gleich?	**124** s	31 s	4,0
Wo Zeit einbezogen?	45 s	11 s	4,1
Falsche Antworten	**4**/15	**0**/15	

Faktor vier durchgehend — und die letzte Zeile wiegt schwerer als alle Zeiten: Bei Fassung A war jede vierte Antwort falsch, und die Befragten waren sich ihrer Sache sicher. Ein Code, den man falsch versteht, ist gefährlicher als einer, den man nicht versteht.

Der Aufwand: Eine Stunde für drei Personen. Damit ist jede Stildiskussion entscheidbar — und die Erfahrung zeigt, dass sie oft anders ausgeht, als der Verfasser erwartet.

(c) Was das Lesen erschwert.

Eigenschaft	Wirkung	Behebbar?
Namen ohne Bedeutung	**sehr hoch**	leicht
Weite Sprünge im Aufruf	hoch	Umbau
Zustand über Dateigrenzen	**sehr hoch**	Umbau
Bedingung ohne Begründung	hoch	**Kommentar**
Auskommentierter Code	hoch	löschen
Uneinheitliche Schreibweise	mittel	Formatierer
Tote Zweige	hoch	löschen

Drei der sieben sind in Minuten behoben — auskommentierten Code löschen, tote Zweige löschen, Formatierer laufen lassen. Zusammen machen sie einen erheblichen Teil der Leseschwierigkeit aus und kosten nichts.

Die vierte Zeile im Detail.

```
# schlecht:  sagt, was ohnehin dasteht
if b.abnahme < grenze:  # wenn vor der Grenze

# gut:  sagt, warum
if b.abnahme < grenze:
# Fall 4711:  Altsystem lieferte 1970 als Datum,
# wenn die Abnahmezeit fehlte.  Nicht loeschen.
```

Der zweite Kommentar beantwortet die Frage, die der Code nicht beantworten kann — und er verhindert, dass ein Nachfolger die Bedingung als überflüssig entfernt. Genau dafür sind Kommentare da, und nur dafür.

Man versteht Code, indem man ihn ändert

Vorgehen	Verständnis	
Lesen	oberflächlich	**man glaubt zu verstehen**
Erklären können	gut	
Ändern und prüfen	**belastbar**	**die Prüfung antwortet**

Die wirksamste Leseübung ist eine kleine, gefahrlose Änderung mit Vorhersage: Man sagt vorher, was sich ändern wird, und prüft es. Wer richtig vorhersagt, hat verstanden; wer falsch liegt, hat gerade das Wichtigste gelernt.

Das ist dasselbe Verfahren wie in Kapitel 7: eine Hypothese bilden und sie einer Prüfung aussetzen, die sie widerlegen könnte. Lesen und Fehlersuchen sind dieselbe Tätigkeit mit verschiedenem Anlass.

Spinellis[25] widmet dem Lesen ein eigenes Buch — das einzige seiner Art; sein Folgeband[26] behandelt die Eigenschaften, an denen man Qualität erkennt; Knuth[13] forderte früh, Programme für Leser statt für Übersetzer zu schreiben.

9 Prüfen

Prüfungen sind ein Entwurfswerkzeug

Zweck	Bekannt?	
Fehler finden	ja	
Änderungen absichern	ja	
Den Entwurf prüfen	**selten genannt**	**der interessante**
Dokumentation	teils	

Wenn eine Prüfung mühsam zu schreiben ist, liegt das fast nie an der Prüfung. Es liegt daran, dass die geprüfte Funktion zu viel gleichzeitig tut, zu viel Umgebung braucht oder ihre Zuständigkeit nicht klar hat. Die Mühe ist eine Rückmeldung über den Entwurf.

Die Rückmeldung lesen.

Aufbau dauert lange	Zu viel Umgebung — Kapitel 6
Viele Attrappen nötig	**Zu viele Abhängigkeiten**
Prüfung ist unverständlich	Die Funktion tut mehrere Dinge
Bricht bei Umbauten	**Gegen die innere Struktur geprüft**

Die letzte Zeile ist die häufigste Ursache dafür, dass Prüfbestände sterben: Sie prüfen, wie *etwas gemacht wird, statt* was *herauskommt. Dann bricht bei jeder Umgestaltung die halbe Suite, und irgendwann schaltet man sie ab.*

Was man prüft und was nicht

Gegenstand	Prüfen?	
Fachliche Regeln	**unbedingt**	**tragen Entscheidungen**
Grenzfälle	unbedingt	dort sitzen Fehler
Zusammenspiel an Rändern	ja	wenige
Triviale Zuweisungen	nein	Aufwand ohne Ertrag
Fremdbibliotheken	nein	nicht unsere Aufgabe

Die vorletzte Zeile widerspricht der Abdeckungslogik: Wer hundert Prozent anstrebt, prüft auch Zuweisungen und Weiterreichungen — und verbringt die Hälfte der Zeit mit Prüfungen, die nie fehlschlagen. Band 17, Kapitel 7 zeigt die Ertragskurve.

Eigenschaften statt Einzelfälle

	Einzelfall	Eigenschaft
Man schreibt	**Eingabe und Ergebnis**	**eine Regel, die gilt**
Deckt ab	einen Fall	Hunderte
Findet Grenzfälle	nur bedachte	**auch unbedachte**
Bei Fehlschlag	der Fall	das *kleinste* Gegenbeispiel

Beispiel aus unserem Fall: „Die Medikation zu einem Zeitpunkt enthält nie zwei Anordnungen desselben Wirkstoffs.“ Das ist eine Regel über alle möglichen Anordnungsmengen — und ein Werkzeug erzeugt Hunderte davon und schrumpft ein Gegenbeispiel auf die kleinste Form.

Aufgabe 9.1.

(a) Prüfen Sie die Anlassbestimmung.
(b) Formulieren Sie Eigenschaften.
(c) Bestimmen Sie, wo Attrappen schaden.

Lösung Schritt für Schritt.

(a) Die Anlassbestimmung prüfen.
Die Grenzfälle, geordnet.

Fall	Erwartet	Art
Kein Vorbefund	**keine Änderung**	Rand
Wert genau auf Grenze	**nicht außerhalb**	**Rand**
Grenzwert `< 0.1`	leere Menge	Typ
Ungültige Probe	nur `UNGUELTIG`	Vorrang
Kritisch und Änderung	**beide Anlässe**	Mehrfach
Zwei gleiche Werte	keine Änderung	

Sechs Fälle, und fünf davon liegen an Rändern. Das ist kein Zufall — die Mitte des Wertebereichs ist fast immer richtig behandelt, weil man sie beim Schreiben vor Augen hat. Die Ränder sieht man erst, wenn man nach ihnen sucht.

Die Prüfung als Text.

```
def test_wert_genau_auf_grenze_ist_nicht_aussen():
b = befund(kalium, wert=3.5, bereich=(3.5, 5.1))
assert Anlass.NEU_AUSSERHALB not in anlaesse(b, None)
```

Der Name ist die halbe Prüfung: Er sagt, welche Regel gilt, und erscheint bei einem Fehlschlag im Protokoll. Ein Test namens `test_3` zwingt dazu, den Code zu lesen, um zu erfahren, was kaputt ist.

Die Bilanz.

	Zeilen	Dauer	Gefundene Fehler
Sechs Grenzfälle	34	2 ms	**3**
Zwanzig Regelfälle	96	6 ms	0

Sechs Randfälle finden drei Fehler, zwanzig Regelfälle keinen. Das ist die übliche Verteilung und die Begründung für die Reihenfolge: Ränder zuerst, Mitte danach — und die Mitte oft gar nicht.

(b) Eigenschaften formulieren.

Eigenschaft	Gegenstand	Findet
Verlauf ist aufsteigend	Sortierung	Vergleichsfehler
Verlauf ⊆ Eingabe	Auswahl	**erfundene Werte**
Ersetzte nie enthalten	Auswahl	
Keine zwei Anordnungen desselben Wirkstoffs	Medikation	**Überlappungen**
Bestätigte nie offen	Warnungen	

Die vierte Zeile ist die wertvollste, weil sie eine fachliche Regel ausdrückt, die niemand als Einzelfall geprüft hätte. Ein Erzeuger, der zufällige Anordnungsfolgen baut, findet Überlappungen, die im Alltag selten und in ihrer Wirkung erheblich sind.

Was ein Erzeuger können muss.

Realistische Struktur	Zeitpunkte in plausibler Reihenfolge, Werte im plausiblen Bereich
Randfälle einstreuen	**Grenzwerte, leere Mengen, gleiche Zeitpunkte**
Reproduzierbar	**Bei Fehlschlag mit demselben Startwert wiederholbar**

Die dritte Zeile ist unverzichtbar und wird oft vergessen: Ein Eigenschaftstest, der bei jedem Lauf andere Daten erzeugt und deren Startwert nicht ausgibt, meldet einen Fehler, den man nie wieder herstellen kann — Kapitel 7, Schritt eins, verletzt.

(c) Wo Attrappen schaden.

Ersetzt wird	Sinnvoll?	
Netzaufruf	**ja**	langsam, unzuverlässig
Uhrzeit	ja	**besser: Argument**
Datenbank	teils	echte ist genauer
Eigene Fachlogik	**nein**	**man prüft die Attrappe**

Die letzte Zeile beschreibt den häufigsten Missbrauch: Eine Prüfung, in der alle Aufgerufenen durch Attrappen ersetzt sind, prüft nur noch, dass die Aufrufe in einer bestimmten Reihenfolge geschehen — also die innere Struktur. Sie bricht bei jedem Umbau und findet nie einen echten Fehler.

Die Alternative.

Statt Attrappe	Aufwand	Wirkung
Reine Funktion ohne Ränder	**keiner**	**Kap. 6**
Einfache Ersatzumsetzung	gering	prüft Verhalten
Echte Datenbank im Speicher	mittel	prüft das Echte

Die erste Zeile ist die Auflösung: Wenn die Fachlogik keine Ränder berührt, braucht sie keine Attrappen. Der Bedarf an Attrappen ist selbst ein Maß dafür, wie gut die Trennung aus Kapitel 6 gelungen ist.

Der Prüfbestand ist Code

Eigenschaft	Gilt?	
Muss gepflegt werden	ja	
Kann Fehler enthalten	ja	**grüne Prüfung ohne Zusicherung**
Wird gelesen	ja	**öfter als der Code**
Umfang	1–2× Produktivcode	

Die dritte Zeile ist der unterschätzte Punkt: Wer eine fremde Funktion verstehen will, liest zuerst ihre Prüfungen — sie zeigen die vorgesehenen Fälle und die Randbedingungen. Ein gut benannter Prüfbestand ist die beste Dokumentation, die es gibt, weil sie nicht veralten kann.

Das ist zugleich ein Lesezugang, der in Kapitel 8 fehlt: Bei einem unbekannten System sind die Prüfungen oft der schnellste Weg zu der Frage, was das System eigentlich garantiert.

Okken[18] behandelt die praktischen Werkzeuge für Prüfungen in Python; McConnell[16] sammelt die Befunde dazu, wo Fehler tatsächlich sitzen; Hunt und Thomas[10] behandeln Prüfbarkeit als Entwurfseigenschaft.

Part IV

Python im Besonderen

10 Was Python anders macht

Protokolle statt Vererbung

	Vererbung	Protokoll
Bindung	**an eine Hierarchie**	**an ein Verhalten**
Nachträglich	schwer	möglich
Fremde Typen	nein	**ja**
Prüfbar	Übersetzer	Typprüfer, optional

Wer in Python eine Klassenhierarchie baut, um Gemeinsamkeiten auszudrücken, schreibt ein Programm, das genauso gut in einer anderen Sprache stünde. Die eigene Ausdrucksform ist das Protokoll: Was zählt, ist, welche Operationen ein Objekt unterstützt — nicht, von wem es abstammt.

An unserem Fall.

```
class Bewertbar(Protocol):
def aussen(self, wert:  Ergebnis) -> bool:  ...
def kritisch(self, wert:  Ergebnis) -> bool:  ...
```

Damit kann ein Referenzbereich aus dem Labor, ein altersabhängiger Bereich und ein individuell festgelegter Bereich dasselbe Protokoll erfüllen, ohne verwandt zu sein — und ein Prüfbestand kann eine eigene Umsetzung einsetzen, ohne von irgendetwas zu erben.

Was den Unterschied ausmacht

Mittel	Wofür	Wird unterschätzt
Erzeugerausdrücke	**Ströme ohne Speicher**	stark
Kontextverwalter	**Aufräumen garantiert**	stark
Datenklassen	Werte statt Objekte	
Zuordnungsmuster	Fallunterscheidung über Typen	stark
Typangaben	**Absicht dokumentieren**	sehr stark

Die letzte Zeile ist die wichtigste und wird am häufigsten mit „Python ist doch dynamisch“ abgetan. Typangaben ändern das Laufzeitverhalten nicht — sie machen die Absicht prüfbar. Bei zehntausend Zeilen ist das der Unterschied zwischen änderbar und nicht änderbar.

Der Zen und was er wirklich sagt.

Ausdrücklich	Besser als versteckt
Einfach	Besser als verwickelt
Flach	Besser als geschachtelt
Fehler	**Sollen niemals stillschweigend vergehen**

Die letzte Zeile ist die praktisch folgenreichste. Ein `except: pass` *verletzt sie — und ist in Bestandssystemen die häufigste Ursache für Fehler, die niemand bemerkt, bis die Daten falsch sind.*

Wo Python zur Falle wird

Erscheinung	Folge	
Veränderliche Vorgabewerte	**geteilt über Aufrufe**	klassische Falle
Späte Bindung in Abschlüssen	alle sehen den letzten Wert	
`is` statt ==	funktioniert zufällig	**kleine Zahlen**
Gleitkomma für Geld	**Rundungsfehler**	Kap. 2

Alle vier funktionieren in den kleinen Fällen, mit denen man sie ausprobiert, und brechen in größeren. Die dritte ist besonders tückisch: `a is b` liefert für kleine ganze Zahlen `True`, weil sie zwischengespeichert werden — und für große `False`.

Aufgabe 10.1.

(a) Schreiben Sie eine Auswertung als Strom.
(b) Nutzen Sie Zuordnungsmuster für die Fallunterscheidung.
(c) Bestimmen Sie den Nutzen von Typangaben.

Lösung Schritt für Schritt.

(a) Eine Auswertung als Strom.
Die Aufgabe. Aus einer Datei mit 4 Millionen Befundzeilen die auffälligen eines Patienten.
Erster Entwurf: alles laden.

```
zeilen = open(pfad).readlines()        # 4 Mio
befunde = [parse(z) for z in zeilen]   # 4 Mio
meine = [b for b in befunde if b.patient == p]
return [b for b in meine if anlaesse(b, None)]
```

Zweiter Entwurf: als Strom.

```
def befunde_aus(pfad):
with open(pfad) as f:
for zeile in f:
yield parse(zeile)

meine = (b for b in befunde_aus(pfad) if b.patient
== p)
return [b for b in meine if anlaesse(b, None)]
```

	Laden	**Strom**	
Speicher	**3,2 GB**	**12 MB**	270×
Erste Ausgabe nach	95 s	**0,3** s	
Gesamtdauer	112 s	88 s	
Abbruch möglich	nein	**ja**	

Der Speichergewinn ist der offensichtliche; die zweite Zeile ist der wichtigere. Bei einem Strom erscheint das erste Ergebnis nach drei Zehntelsekunden — der Benutzer sieht sofort etwas. Band 18, Kapitel 8 beziffert, was das wert ist.

Der Preis: Ein Erzeuger lässt sich nur einmal durchlaufen. Wer zweimal darüber gehen will, muss ihn neu erzeugen oder doch sammeln — und das ist der häufigste Fehler beim Umstieg.

(b) Zuordnungsmuster.
Der Fall. Die Fallunterscheidung über den Ergebnistyp aus Kapitel 2.

```
def als_text(e:  Ergebnis, einheit:  str) -> str:
match e:
case Zahlwert(betrag):
return f"{betrag} {einheit}"
case Grenzwert(richtung, schranke):
return f"{richtung}{schranke} {einheit}"
case Qualitativ(befund):
return befund
case Nichtauswertbar(grund):
return f"n. a. ({grund})"
```

	`isinstance`-Kette	Zuordnungsmuster
Zeilen	14	**10**
Zerlegung	von Hand	**im Muster**
Vergessener Fall	stiller `None`	Typprüfer meldet

Die letzte Zeile ist der eigentliche Gewinn und funktioniert nur zusammen mit Typangaben: Ein Typprüfer erkennt, dass nicht alle Fälle behandelt sind, wenn der Rückgabetyp `str` verlangt und ein Zweig fehlt. Ohne Typangaben bleibt der vergessene Fall still.

(c) Der Nutzen von Typangaben.

Systemgröße	Aufwand	Gefundene Fehler	Lohnt?
$<$ 500 Zeilen	gering	$\approx$ 0	nein
5000 Zeilen	mittel	$\approx$ 8	knapp
50 000 Zeilen	mittel	$\approx$ 90	**ja**
Mit fremden Lesern	—	—	**unbedingt**

Die letzte Zeile ist unabhängig von der Größe: Sobald jemand anderes den Code liest, ist eine Typangabe die knappste Form der Dokumentation — sie steht an der Signatur, veraltet nicht und wird geprüft. Kapitel 8 beziffert, was das beim Lesen spart.

Was Typangaben nicht leisten.

Erwartung	Erfüllt?	
Fehler zur Laufzeit verhindern	**nein**	**sie werden nicht geprüft**
Geschwindigkeit	nein	
Fachliche Richtigkeit	nein	Band 19, Kap. 8
Absicht festhalten	**ja**	

Die erste Zeile überrascht Umsteiger: Python prüft Typangaben zur Laufzeit nicht. Sie wirken nur, wenn ein Prüfer läuft — und der gehört in die Prüfstrecke, sonst sind sie Kommentare, die veralten.

Pythonisch heißt nicht kurz

Missverständnis	Tatsächlich	
Möglichst wenig Zeilen	nein	**Kapitel 4**
Möglichst clever	nein	
Alle Sprachmittel nutzen	nein	
Ausdrücklich und einfach	**ja**	steht im Zen

Ein verschachtelter Erzeugerausdruck über drei Ebenen ist Python und nicht pythonisch. Der Maßstab steht in der letzten Zeile und ist derselbe wie in Kapitel 4: möglichst wenige Dinge gleichzeitig im Kopf — gemessen, nicht behauptet.

Ramalho[22] erklärt die Protokolle und Datenmodelle, die Python ausmachen; Slatkin[24] sammelt die Einzelregeln mit Begründung; die Sprachvorschläge selbst[23] formulieren die Leitsätze knapper als jedes Lehrbuch.

11 Die Werkzeuge der Sprache

Kontextverwalter: Aufräumen garantiert

	Von Hand	Kontextverwalter
Aufräumen	**man muss daran denken**	**geschieht immer**
Bei Ausnahme	übersprungen	trotzdem
Bei frühem `return`	übersprungen	trotzdem
Sichtbar	verstreut	**ein Block**

Die zweite und dritte Zeile sind der Grund. Ein Aufräumschritt am Ende einer Funktion wird bei jeder Ausnahme und jedem vorzeitigen Verlassen übersprungen — und beides kommt vor. Der Kontextverwalter macht daraus eine Zusicherung.

Ein eigener für unseren Fall.

```
@contextmanager
def messung(name):
start = perf_counter()
try:
yield
finally:
dauer = perf_counter() - start
log.info("dauer", schritt=name, ms=dauer*1000)
```

Acht Zeilen, und die Zeitmessung erscheint auch dann im Protokoll, wenn der gemessene Block mit einer Ausnahme endet — was gerade bei Leistungsproblemen der interessante Fall ist. Kapitel 12 benutzt ihn.

Dekoratoren: mit Maß

Verwendung	Sinnvoll?	
Zwischenspeichern	ja	aber: Kap. 6
Protokollieren	ja	querschneidend
Wiederholen bei Fehler	ja	
Fachlogik	**nein**	**versteckt sie**
Rechteprüfung	umstritten	**leicht zu vergessen**

Ein Dekorator fügt Verhalten hinzu, das beim Lesen der Funktion nicht sichtbar ist — das ist sein Zweck und seine Gefahr. Für Querschnittsbelange ist das richtig; für Fachlogik ist es das Gegenteil von „ausdrücklich besser als versteckt“.

Die letzte Zeile ist der Fall aus Band 19, Kapitel 6: Eine Rechteprüfung als Dekorator ist elegant und wird an einzelnen Funktionen vergessen. Wo sie zwingend ist, gehört sie an die Datenzugriffsschicht, wo sie nicht vergessen werden kann.

Ausnahmen: wofür und wofür nicht

Lage	Ausnahme?	
Datei fehlt	ja	erwarteter Sonderfall
Netz weg	ja	
Ungültige Eingabe	ja	**mit eigener Klasse**
Kein Befund gefunden	**nein**	leere Liste ist keine Ausnahme
Programmfehler	nein	**Zusicherung, Band 17**
Ablaufsteuerung	**nein**	unlesbar

Die vierte Zeile ist ein häufiger Entwurfsfehler: „Nichts gefunden“ ist ein normales Ergebnis, keine Ausnahme — und wer es zur Ausnahme macht, zwingt jeden Aufrufer zu einem Fangblock für den Regelfall.

Aufgabe 11.1.

(a) Bauen Sie einen Erzeuger für den Verlauf.
(b) Entwerfen Sie die Ausnahmehierarchie.
(c) Bestimmen Sie, wann ein Dekorator schadet.

Lösung Schritt für Schritt.

(a) Ein Erzeuger für den Verlauf.
Die Aufgabe. Werte paarweise durchlaufen, um Änderungen zu bestimmen.

```
def paare(folge):
"""Liefert (vorheriger, aktueller); erstes Paar (None,
x0)."""
vorher = None
for x in folge:
yield vorher, x
vorher = x

for v, b in paare(verlauf(...)):
for a in anlaesse(b, v):
melde(b, a)
```

Sechs Zeilen, und sie beseitigen eine ganze Fehlerklasse: das Verwalten eines Vorgängerzeigers von Hand. Der erste Durchlauf liefert `None` als Vorgänger — der Randfall ist im Erzeuger behandelt statt an jeder Aufrufstelle.

Die Bilanz.

	Von Hand	Erzeuger	
Zeilen je Aufrufstelle	5	**2**	
Randfall behandelt	je Stelle	**einmal**	
Aufrufstellen	7	7	
Fehler gefunden	—	**2**	Randfall vergessen

Zwei der sieben Stellen hatten den ersten Durchlauf falsch behandelt — sie verglichen den ersten Wert mit sich selbst. Das ist der typische Ertrag: Ein herausgezogener Randfall deckt auf, wo er bisher falsch behandelt wurde.

(b) Die Ausnahmehierarchie.

```
class KisFehler(Exception):
"""Basis aller eigenen Fehler."""

class DatenFehler(KisFehler):  ...
class BefundUnvollstaendig(DatenFehler):  ...
class EinheitUnbekannt(DatenFehler):  ...

class ZugriffFehler(KisFehler):  ...
class NichtBerechtigt(ZugriffFehler):  ...
```

Eigene Basis	Erlaubt „alles Eigene fangen“ ohne fremde Fehler zu schlucken
Zwei Ebenen	**Grob fangen an den Rändern, fein wo man handeln kann**
Kein `except Exception`	fängt auch Programmfehler

Die dritte Zeile ist die wichtigste Regel: Ein weiter Fangblock schluckt auch Tippfehler in Attributnamen und Typverwechslungen — also genau die Fehler, die laut werden sollen. Das ist die Verletzung von „Fehler sollen niemals stillschweigend vergehen“.

Was in eine Ausnahme gehört.

Angabe	**Nötig?**	
Was fehlgeschlagen ist	**ja**	
Welche Daten	**ja**	**Kennung, nicht Inhalt**
Was zu tun ist	wenn bekannt	
Patientendaten	**nein**	**landet im Protokoll**

Die letzte Zeile ist eine Regel aus Band 19: Fehlermeldungen wandern in Protokolle, Protokolle wandern in Auswertungssysteme, und die haben andere Zugriffsrechte. Eine Ausnahme nennt Kennungen, nie Inhalte.

(c) Wann ein Dekorator schadet.
Der Fall. Zwischenspeichern der Referenzbereiche.

```
@lru_cache(maxsize=None)
def bereich_fuer(code, alter, geschlecht):
return db.hole_bereich(code, alter, geschlecht)
```

Wirkung	Bewertung	
Schneller	ja	$\approx 400\times$
Änderung im Labor	**wirkt nicht**	**bis Neustart**
Speicher	unbegrenzt	`maxsize=None`
In Prüfungen	**Ergebnisse hängen ab**	Reihenfolge zählt

Die zweite Zeile ist der klinisch heikle Fall: Das Labor ändert einen Referenzbereich, und die Anwendung bewertet weiter nach dem alten — bis jemand neu startet. Der Zwischenspeicher ist unsichtbar, und deshalb sucht niemand dort.

Der zweite Entwurf.

Maßnahme	Wirkung	
Ablaufzeit setzen	begrenzt	**z. B.** 5 **min**
Größe begrenzen	Speicher	
Abschaltbar	**Fehlersuche**	**Kap. 6**
Am Rand statt in der Logik	**Fachlogik bleibt rein**	

Der Zwischenspeicher bleibt — vierhundertfacher Gewinn gibt man nicht auf. Er bekommt nur eine Ablaufzeit, eine Grenze und einen Schalter. Drei Zeilen mehr, und aus einer verborgenen Fehlerquelle wird ein beherrschtes Werkzeug.

Sprachmittel sind kein Selbstzweck

Frage vor dem Einsatz	Antwort
Löst es ein Problem, das ich habe?	muss ja sein
Versteht es der nächste Leser?	muss ja sein
Wäre es ohne einfacher?	**dann ohne**

Metaklassen, Deskriptoren und eigene Zuordnungsprotokolle sind mächtig und lösen Probleme, die die meisten Programme nicht haben. Wer sie einsetzt, weil er sie kann, erhöht die Verstehenszeit für alle Nachfolger — und die ist messbar, wie Kapitel 8 zeigt.

Ramalho[22] behandelt die Sprachmittel mit ihren Abwägungen; Hettinger[9] zeigt in seinen Vorträgen den Unterschied zwischen möglich und angemessen; Beazley und Jones[2] sammeln die Anwendungsfälle, in denen sie sich lohnen.

12 Wo Python weh tut

Drei Grenzen

Grenze	Wirkung	Ausweg
Geschwindigkeit	$\approx$ 30–100$\times$ langsamer als C	**Bibliotheken, Vektoren**
Fäden	**keine echte Parallelität**	**Prozesse**
Typen zur Laufzeit	Fehler spät	Typprüfer

Alle drei sind bekannt und werden in beide Richtungen übertrieben. Die erste gilt für Schleifen in reinem Python und nicht für Arbeit, die in Bibliotheken stattfindet — und die meiste Rechenarbeit findet dort statt.

Wo die Zeit tatsächlich hingeht.

Anteil	Typisch	
Datenbankabfragen	$\approx$ 55 %	**oft** $n{+}1$
Netz und Serialisierung	$\approx$ 20 %	
Reines Python	$\approx$ 15 %	
Bibliotheken	$\approx$ 10 %	

Fünfzehn Prozent der Zeit vergehen in dem Teil, den man beschleunigen kann, indem man die Sprache wechselt. Wer eine Anwendung wegen der Geschwindigkeit neu schreibt, ohne die erste Zeile geprüft zu haben, optimiert die falsche Größe.

Das $n+1$-Problem

```
# eine Abfrage fuer die Faelle ...
for fall in hole_faelle(patient):
# ...  und eine je Fall
for b in hole_befunde(fall.id):
zeige(b)
```

Fälle	Abfragen	Dauer	
1	2	8 ms	
12	13	52 ms	
80	81	324 ms	spürbar
340	**341**	**1,4 s**	Langzeitpatient

Beim Entwickeln mit drei Testfällen ist nichts zu bemerken. Beim Patienten mit dreihundertvierzig Fällen dauert der Aufbau anderthalb Sekunden — und das ist genau der Patient, bei dem es auf Geschwindigkeit ankommt.

Messen, nicht raten

Vorgehen	Trefferquote	
Raten, wo es langsam ist	≈ 20 %	**meist falsch**
Profil erstellen	100 %	**zeigt es**
Abfragen zählen	100 %	findet $n+1$

Die erste Zeile ist gut belegt und gilt auch für erfahrene Entwickler: Die Vermutung, wo ein Programm langsam ist, trifft in etwa einem von fünf Fällen zu. Ein Profillauf dauert fünf Minuten und beendet die Diskussion.

Aufgabe 12.1.

(a) Beheben Sie das $n+1$-Problem.

(b) Vergleichen Sie Fäden und Prozesse.

(c) Bestimmen Sie, wann man die Sprache verlässt.

Lösung Schritt für Schritt.

(a) Das $n{+}1$-Problem beheben.

```
fallnummern = [f.id for f in hole_faelle(patient)]
alle = hole_befunde_zu(fallnummern)     # eine Abfrage
nach_fall = defaultdict(list)
for b in alle:
nach_fall[b.fall].append(b)
```

Fälle	Vorher	Nachher	Faktor
12	52 ms	11 ms	4,7
80	324 ms	18 ms	18
340	**1,4 s**	**34 ms**	**41**

Faktor einundvierzig beim schwersten Fall, Faktor fünf beim leichten — und der Gewinn wächst mit der Belastung. Das ist das Kennzeichen einer echten Verbesserung: Sie wirkt dort am stärksten, wo es weh tut.

Die Grenze.

Fallnummern	Eine Abfrage?	
Bis ≈ 1000	ja	
> 1000	**Grenze der Abfrage**	**in Blöcken**
$> 10^5$	nein	Verknüpfung statt Liste

Auch der zweite Entwurf hat eine Grenze — und wer sie nicht kennt, ersetzt ein $n{+}1$-Problem durch eine Abfrage, die an der Längenbeschränkung scheitert. Größenordnungen kennen, wie in Kapitel 3.

(b) Fäden gegen Prozesse.

Aufgabe	Fäden	Prozesse
Warten auf Netz	**gut**	unnötig teuer
Warten auf Platte	gut	
Rechnen	**kein Gewinn**	**echter Gewinn**
Daten teilen	einfach	Serialisierung
Fehlerklassen	Wettläufe	getrennt

Die dritte Zeile ist die bekannte Einschränkung: Bei reiner Rechenarbeit bringen Fäden in Python nichts, weil nur einer zur Zeit rechnet. Beim Warten dagegen — und das ist in einem KIS die überwiegende Tätigkeit — bringen sie den vollen Gewinn.

Die Messung an unserem Fall.

Aufgabe	Einzeln	8 **Fäden**	Faktor
40 Laborabfragen	3,2 s	**0,5** s	**6,4**
Verlauf berechnen	1,8 s	1,9 s	0,95

Die zweite Zeile ist minimal langsamer — die Verwaltung der Fäden kostet, und der Gewinn ist null. Wer Fäden auf Rechenarbeit ansetzt, macht das Programm komplizierter und um fünf Prozent langsamer.

(c) Wann man die Sprache verlässt.

Lage	Wechseln?	
Anwendung ist zu langsam	**erst messen**	**meist die Datenbank**
Enge Schleife über 10^8 Werte	Bibliothek	nicht die Sprache
Harte Zeitschranken	ja	Speicher-bereinigung
Systemnahe Arbeit	ja	**Band 21**

Die zweite Zeile ist der häufigste Fall und wird fast immer falsch beantwortet: Eine Schleife über hundert Millionen Werte gehört nicht in eine andere Sprache, sondern in eine vektorisierende Bibliothek — dieselbe Sprache, hundertfache Geschwindigkeit, weil die Schleife dann in C läuft.

Die Reihenfolge.

	Schritt	Aufwand	Typischer Gewinn
1	Messen	1 h	—
2	**Abfragen bündeln**	4 h	5–40×
3	Verfahren verbessern	1–3 d	2–100×
4	Bibliothek einsetzen	2 d	10–100×
5	Teil in C	1–3 Wo.	3–30×

Die Reihenfolge ist nach Aufwand geordnet, und sie wird regelmäßig von hinten begonnen. Schritt zwei kostet einen halben Tag und bringt beim schwersten Fall Faktor einundvierzig — Schritt fünf kostet Wochen für weniger.

Die Sprache ist selten das Problem

Behauptete Ursache	Tatsächlich	
„Python ist zu langsam“	$n{+}1$-Abfragen	**häufigster Fall**
„Die Datenbank ist zu langsam“	fehlender Index	
„Zu viele Daten“	**alles geladen statt gefiltert**	

In allen drei Fällen wird eine Eigenschaft des Werkzeugs beschuldigt, wo eine Entwurfsentscheidung die Ursache ist. Der Unterschied ist wichtig: Die eine Erklärung führt zu einem Neuschreiben in einer anderen Sprache, die andere zu vier Stunden Arbeit.

Das ist keine Verteidigung von Python. Es gibt Aufgaben, für die es die falsche Sprache ist — Band 21 behandelt sie. Aber die Entscheidung braucht eine Messung, und die fehlt fast immer.

Slatkin[24] behandelt Leistungsfragen mit Messungen statt Vermutungen; Martelli und Mitautoren[15] beschreiben die Laufzeiteigenschaften im Einzelnen; Norvig[17] zeigt an durchgerechneten Beispielen, wie weit man mit besseren Verfahren statt schnellerer Sprache kommt.

Part V

Der zweite Entwurf

13 Umschreiben

Die Bedingung

Umgestalten heißt	Struktur ändern, Verhalten erhalten
Die Voraussetzung	**Ein Prüfnetz, das Verhaltensänderungen meldet**
Ohne Prüfnetz	Kein Umgestalten, sondern Umschreiben auf Verdacht

Die zweite Zeile ist nicht verhandelbar. Wer ohne Prüfungen umbaut, ändert das Verhalten und merkt es nicht — und in einem klinischen System kann diese Änderung eine Warnung sein, die nicht mehr erscheint.

Die Reihenfolge bei Altbestand.

1	Die Stelle finden, an der geändert werden muss
2	**Einen festhaltenden Test schreiben** — er prüft das *jetzige* Verhalten
3	Umgestalten, in kleinen Schritten, jeder für sich lauffähig
4	Erst dann die eigentliche Änderung

Schritt 2 ist der ungewöhnliche: Man prüft nicht, ob das Verhalten richtig *ist, sondern hält fest, wie es* ist *— auch wenn es falsch sein sollte. Damit schützt man sich vor unbeabsichtigten Änderungen, ohne die Fachlogik verstanden haben zu müssen.*

Der Fall: die Verlaufsansicht

Der Bestand. Eine Funktion von 140 Zeilen aus Kapitel 5, gewachsen über vier Jahre.

Schr.	Was	Zeilen ber.	Lauffähig?
1	Festhaltender Test	0	ja
2	**Zeit als Argument**	6	ja
3	Beschaffung herausziehen	22	ja
4	Darstellung herausziehen	55	ja
5	**Fachlogik bleibt — nun rein**	63	ja
6	$\mathcal{O}(n^2)$ beseitigen	9	ja
7	Anlässe als Menge	18	ja

Sieben Schritte, jeder einzeln lauffähig und einzeln übergebbar. Das ist der entscheidende Unterschied zur großen Umbauaktion: Man kann jederzeit aufhören, und der Stand ist brauchbar — Band 17, Kapitel 13.

Die Bilanz.

	Vorher	Nachher	
Zeilen gesamt	140	152	+9 %
Prüfbare Zeilen	0	**63**	
Verstehenszeit	210 s	**55** s	3,8×
Dauer bei 50 000	112 s	**48** ms	
Prüffälle	2	**34**	

Neun Prozent mehr Zeilen — der zweite Entwurf ist nicht kürzer. Er ist prüfbar, viermal schneller zu verstehen und zweitausendmal schneller im schwersten Fall. Wer Zeilen zählt, sieht eine Verschlechterung.

Der festhaltende Test

```
def test_verlauf_unveraendert():
ein = lade("daten/patient_4711.json")
erg = zeige_verlauf(ein, KALIUM, 30)
assert erg == lade("daten/erwartet_4711.json")
```

Die erwartete Ausgabe wird beim ersten Lauf erzeugt und gespeichert — man beurteilt sie nicht, man friert sie ein. Der Test sagt nicht „das Ergebnis ist richtig“, sondern „das Ergebnis ist unverändert“, und genau das braucht man beim Umbau.

Die Gefahr: Wenn das eingefrorene Verhalten falsch war, zementiert der Test den Fehler. Deshalb gehört er zum Umbau und wird danach durch echte Prüfungen ersetzt — er ist ein Gerüst, kein Bauteil.

Aufgabe 13.1.

(a) Führen Sie einen Umbau in kleinen Schritten durch.
(b) Bestimmen Sie, wo umgebaut werden soll.
(c) Untersuchen Sie, wann Neuschreiben besser ist.

Lösung Schritt für Schritt.

(a) Umbau in kleinen Schritten.
Schritt 2 im Einzelnen. Die Zeit zum Argument machen.

	Teilschritt	Dauer	Prüfung
a	Argument mit Vorgabe `None`	10 min	grün
b	`now()` nur bei `None`	5 min	grün
c	Aufrufer einzeln umstellen	40 min	grün
d	**Vorgabe entfernen**	10 min	**grün**

Vier Teilschritte statt einem. Nach jedem läuft alles — man kann übergeben, in Urlaub gehen oder abbrechen. Der Kunstgriff steckt in Schritt a: Ein Argument mit Vorgabewert ändert keinen einzigen Aufrufer und schafft trotzdem den neuen Weg.

Warum das wichtiger ist, als es aussieht.

Vorgehen	Zwischen-stände	Abbruch möglich	Risiko
Ein großer Schritt	0	**nein**	hoch
Vier Teilschritte	**4**	**jederzeit**	gering

Dasselbe Argument wie bei der Ablösung in Band 17: Nicht die Zerlegung senkt das Risiko, sondern die Umkehrbarkeit jedes Teils. Vier Schritte ohne Rückweg wären schlechter als einer.

(b) Wo umgebaut werden soll.

Datei	Komplexität	Änderungen	Rang
`verlauf.py`	84	340	**28 560**
`export.py`	**210**	12	2520
`anlass.py`	31	290	8990
`druck.py`	150	95	14 250

`export.py` ist die verworrenste Datei und steht auf Rang vier — sie wird kaum geändert. `verlauf.py` ist nur mittelmäßig verwickelt und steht oben, weil sie ständig angefasst wird. Das Produkt beider Größen ist die richtige Reihenfolge.

Die Zahlen stammen aus der Versionsgeschichte und kosten eine halbe Stunde. Sie ersetzen die Diskussion darüber, welcher Code am schlimmsten ist — eine Diskussion, die sonst nach Geschmack entschieden wird.

Die Wirkung gezielten Vorgehens.

Vorgehen	Aufwand	Wirkung
Alles aufräumen	400 h	100 %
Nach Komplexität	120 h	≈ 35 %
Nach Rang	**120 h**	≈ **72** %

(c) Wann Neuschreiben besser ist.

Lage	Neu?	
Fachlogik unbekannt	**nein**	**sie steckt im Code**
Struktur schlecht	nein	umbauen
Datenmodell falsch	prüfen	teuerster Fall
Kleiner Teil, klar begrenzt	ja	
Prototyp	**ja**	**war dafür gedacht**

Die erste Zeile ist die entscheidende: Ein gewachsenes System enthält Jahre gelöster Sonderfälle, und die stehen nirgends aufgeschrieben. Wer neu schreibt, lernt sie einzeln wieder — durch Fehler im Betrieb.

Der Vergleich am Fall.

	Umbau	Neu
Aufwand	**18** h	60 h
Verhalten gleich	**geprüft**	Hoffnung
Sonderfälle erhalten	ja	**einzeln neu lernen**
Zwischenstände	7	0

Bei einer Funktion von hundertvierzig Zeilen ist der Umbau dreimal billiger und sicherer. Die Rechnung kippt erst, wenn das Datenmodell falsch ist — dann ändert man nicht die Funktion, sondern die Grundlage, und das ist ein anderes Vorhaben.

Umgestalten und Ändern nie gleichzeitig

	Getrennt	Gemischt
Bei Fehlschlag	**Ursache klar**	**welche Änderung?**
Durchsicht	einfach	unmöglich
Rücknahme	gezielt	alles oder nichts

Eine Übergabe, die Struktur und *Verhalten ändert, ist nicht durchsehbar: Der Prüfende muss aus fünfhundert geänderten Zeilen die drei heraussuchen, die etwas anderes tun. Das gelingt nicht, und deshalb werden solche Übergaben durchgewinkt.*

Die Regel lautet: erst umgestalten, übergeben, prüfen — dann ändern. Zwei Übergaben statt einer, und beide sind in Minuten zu beurteilen statt in Stunden.

Fowler[7] liefert den Katalog der Umgestaltungsschritte; Beck[3] beschreibt die kleinen Entscheidungen, aus denen sich Lesbarkeit ergibt; Gamma und Mitautoren[8] stellen das Vokabular bereit, mit dem man über Zielformen spricht.

14 Wann aufhören

Die schwierigste Entscheidung

Zu früh aufhören	Zu spät aufhören	
Unlesbar	**Zeit verbrannt**	
Fehleranfällig	Abstraktion ohne Anlass	
Teuer in der Pflege	**teuer in der Pflege**	**beides**

Die dritte Zeile ist die unangenehme Symmetrie: Beide Fehler führen zu schwer wartbarem Code. Der eine erzeugt Wildwuchs, der andere ein Gerüst aus Abstraktionen, durch das man sich fünf Ebenen hindurchklicken muss, um zu sehen, was geschieht.

Die Abbruchbedingungen.

Verstehenszeit	Ein Fremder beantwortet die üblichen Fragen zügig
Änderbarkeit	**Die nächste erwartete Änderung berührt eine Stelle**
Prüfbarkeit	Die Fachlogik ist ohne Aufbau prüfbar
Größenordnung	Es hält den schwersten realen Fall aus

Vier Bedingungen, alle prüfbar. Wenn sie erfüllt sind, ist der zweite Entwurf fertig — gleichgültig, ob er schön ist. Schönheit ist kein Kriterium, weil sie nicht prüfbar ist und weil sie zwischen zwei Personen verschieden ausfällt.

Die Regel der drei Fälle

Vorkommen	Tun	Warum
Erstes	**schreiben**	
Zweites	doppelt schreiben	noch nicht abstrahieren
Drittes	**jetzt zusammenfassen**	**das Muster ist erkennbar**

Aus einem Fall lässt sich die richtige Verallgemeinerung nicht ablesen, aus zweien selten, aus dreien meistens. Wer beim zweiten Vorkommen abstrahiert, rät — und die falsche Abstraktion ist teurer als die Doppelung, weil sie sich schwerer zurücknehmen lässt.

Warum die falsche Abstraktion so teuer ist.

	Doppelter Code	Falsche Abstraktion
Erkennbar	sofort	**nie**
Rücknahme	leicht	schwer
Wächst durch	Kopieren	**Sonderfall-parameter**
Endzustand	zwei Fassungen	**eine mit acht Schaltern**

Die letzte Zeile beschreibt den Verlauf: Eine verfrüht zusammengefasste Funktion bekommt bei jedem neuen Fall einen Parameter, der ihr Verhalten umschaltet. Nach fünf Jahren hat sie acht davon und niemand versteht mehr, welche Kombinationen zulässig sind.

Der Prüfstein für eine Abstraktion

Gute Abstraktion	Fasst Dinge zusammen, die sich *aus demselben Grund* ändern
Schlechte	**Fasst Dinge zusammen, die zufällig gleich aussehen**
Die Frage	**Wenn sich das eine ändert — muss sich das andere mitändern?**

Zwei Formatierfunktionen, die heute gleich aussehen, sind kein Anlass zur Zusammenfassung, wenn die eine Laborwerte und die andere Abrechnungsbeträge formatiert — sie werden sich aus verschiedenen Gründen auseinanderentwickeln. Gleiches Aussehen ist kein gemeinsamer Grund.

Aufgabe 14.1.

(a) Prüfen Sie die vier Abbruchbedingungen.
(b) Erkennen Sie eine verfrühte Abstraktion.
(c) Bestimmen Sie den Ertrag weiterer Überarbeitung.

Lösung Schritt für Schritt.

(a) Die vier Bedingungen prüfen.
Am Fall aus Kapitel 13.

Bedingung	Prüfung	Ergebnis
Verstehenszeit	5 Fragen, 3 Personen	**55** s gut
Änderbarkeit	**neue Anlassart**	**eine Stelle**
Prüfbarkeit	Fachlogik rein	34 Fälle, 2 ms
Größenordnung	50 000 Befunde	48 ms

Alle vier erfüllt — also aufhören. Es gäbe noch Verbesserungen: Die Beschaffung könnte hinter ein Protokoll, die Darstellung könnte Formate auslagern. Beides wäre eleganter und würde keine der vier Bedingungen verbessern.

Die zweite Bedingung im Einzelnen.

Erwartete Änderung	Berührt	
Neue Anlassart	1 Stelle	gut
Andere Bereichsquelle	1 Stelle	gut
Neue Messgröße	**0** Stellen	**Daten**
Andere Darstellung	1 Stelle	gut

Die dritte Zeile ist das Zeichen für einen gelungenen Entwurf: Eine neue Messgröße erfordert keine Codeänderung, weil sie ein Datum ist. Wer dafür Code ändern muss, hat Daten in den Code geschrieben.

(b) Eine verfrühte Abstraktion erkennen.

```
def verarbeite(daten, modus, mit_filter=True,
sortieren=True, gruppieren=False,
format="lang", grenzen=None,
historisch=False):
```

Anzeichen	Vorhanden?	
Viele Schaltparameter	**ja**	6
Name sagt nichts	ja	`verarbeite`
Unzulässige Kombinationen	**ja**	**nicht dokumentiert**
Aufrufer nutzen Teilmengen	ja	drei Gruppen

Die letzte Zeile enthält die Auflösung: Die Aufrufer zerfallen in drei Gruppen, die je eine feste Parameterkombination verwenden. Das sind drei Funktionen, die verfrüht zu einer zusammengefasst wurden — und die Rücknahme besteht darin, sie wieder zu trennen.

Die Rücknahme.

	Vorher	Nachher	
Funktionen	1	3	
Parameter je Funktion	**7**	**2–3**	
Zeilen gesamt	180	205	+14 %
Zweige	**26**	**9**	−65 %
Verstehenszeit	180 s	**40** s	4,5×

Vierzehn Prozent mehr Zeilen, fünfundsechzig Prozent weniger Zweige — die Doppelung, die man vermeiden wollte, war billiger als die Fallunterscheidung, die sie ersetzt hat.

(c) Der Ertrag weiterer Überarbeitung.

Durchgang	Aufwand	Verstehens-zeit	Gewinn
1 (erster Entwurf)	6 h	210 s	—
2	**18** h	**55** s	−**74** %
3	12 h	48 s	−13 %
4	10 h	46 s	−4 %

Der zweite Durchgang bringt vierundsiebzig Prozent, der dritte dreizehn, der vierte vier. Das ist die übliche Kurve — und der Grund, warum der Band „Der zweite Entwurf“ heißt und nicht „Der vierte“.

Was den dritten Durchgang rechtfertigt.

Anlass	Lohnt?	
Neue Anforderung passt nicht	**ja**	**jetzt weiß man mehr**
Fehler an derselben Stelle	ja	
Es gefällt nicht	**nein**	
Neue Technik verfügbar	nein	

Ein dritter Durchgang braucht einen Anlass von außen — eine Anforderung, die nicht passt, oder wiederholte Fehler. Ohne Anlass ist er Geschmackspflege, und die endet nie von selbst.

Die Kunst ist das Aufhören

Fertigkeit	Lernbar	Verbreitet
Ersten Entwurf schreiben	leicht	überall
Zweiten Entwurf schreiben	übbar	seltener
Erkennen, wann es reicht	**schwer**	**selten**

Die dritte Zeile verlangt, den eigenen Gestaltungswillen gegen einen Maßstab zu stellen, der von außen kommt — die vier Bedingungen vom Kapitelanfang. Wer nur nach dem eigenen Empfinden aufhört, hört entweder zu früh auf oder gar nicht.

Das ist zugleich die Antwort auf die Frage, wovon dieser Band handelt. Nicht davon, wie man guten Code schreibt — sondern davon, woran man erkennt, dass er gut genug ist.

Ousterhout[19] entwickelt die Maßstäbe für Komplexität und ihre Grenzen; Kernighan und Plauger[12] formulierten die Regeln, an denen man Angemessenheit erkennt;

Knuth[14] mahnte früh, dass verfrühte Optimierung — und mit ihr verfrühte Verallgemeinerung — der häufigste Fehler ist.

15 Der Stand im Jahr 2026

Was sich bewährt hat

Praxis	Beleglage	Urteil
Datenmodell zuerst	gut	unstrittig
Fachlogik ohne Zustand	gut	**prüfbar**
Zeit und Umgebung hereinreichen	gut	unstrittig
Unveränderliche Werte	gut	Fehlerklassen weg
Typangaben ab Größe	gut	**Lesbarkeit**
Ränder zuerst prüfen	gut	
Verkleinern bei Fehlersuche	gut	**unterschätzt**
Vollständige Abdeckung	negativ	Aufwand ohne Ertrag
Attrappen für alles	negativ	**prüft sich selbst**
Zeilenzahl als Maß	negativ	misst das Falsche
Frühe Abstraktion	negativ	Kapitel 14

Die obere Gruppe ist unspektakulär und größtenteils vierzig Jahre alt. Die untere besteht aus gut gemeinten Regeln, die zu Zielvorgaben wurden — und damit aufhörten, gute Regeln zu sein.

Fünf Entwicklungen

Erstens: Werkzeuge, die Code vorschlagen.

Wirkung auf	Befund	
Ersten Entwurf	deutlich schneller	bei bekannten Mustern
Zweiten Entwurf	**offen**	**verlangt Urteil**
Fremden Code verstehen	merklich	Kapitel 8
Prüfaufwand	steigt	mehr Code, mehr Durchsicht

Die zweite Zeile ist die Prüffrage dieses Bandes. Ein Werkzeug kann einen ersten Entwurf liefern; ob es den zweiten liefern kann, hängt daran, ob es die vier Bedingungen aus Kapitel 14 beurteilen kann — und die sind an den Zweck gebunden, nicht an den Code.

Die dritte Zeile ist der unerwartete Gewinn und der wichtigste: Wenn Lesen dreißig Prozent der Zeit kostet und ein Werkzeug dabei hilft, ist das mehr wert als jede Beschleunigung des Schreibens.

Zweitens: Typangaben sind Normalfall geworden.

Jahr	Verbreitung	
2015	Randerscheinung	
2020	in Bibliotheken	
2026	**in Anwendungen üblich**	Prüfer in der Strecke

Damit hat Python eine Entwicklung genommen, die viele für ausgeschlossen hielten: Es blieb dynamisch und bekam eine optionale statische Prüfung — Band 17, Kapitel 8 in der Praxis.

Drittens: Die Grenzen verschieben sich.

Grenze	Stand	
Fäden ohne Parallelität	**in Auflösung**	**freischaltbar**
Geschwindigkeit	verbessert	über Jahre stetig
Speicherverbrauch	unverändert	

Die erste Zeile ändert eine Aussage, die dreißig Jahre galt. Die Folgen sind noch nicht absehbar — und die Fehlerklassen aus Kapitel 6 werden dann in Python auftreten, wo sie bisher nicht auftreten konnten.

Viertens: Klinische Software wird reguliert.

Anforderung	Betrifft	
Nachvollzieh-barkeit	**Datenmodell**	Kap. 2
Risikobewertung	Entwurf	
Prüfnachweise	**Prüfbestand**	Kapitel 9

Bemerkenswert ist, dass die Vorschriften genau das verlangen, was dieser Band aus fachlichen Gründen empfiehlt: unveränderliche Daten mit Historie und eine prüfbare Fachlogik. Wer sauber entwirft, erfüllt sie nebenbei.

Fünftens: Auswertende Verfahren in der Medizin.

Voraussetzung	Erfüllt?	
Strukturierte Daten	teils	**Freitext überwiegt**
Einheiten und Bereiche	teils	Kapitel 2
Zeitpunkte sauber	**selten**	**vier Zeiten**
Nachvollzieh-barkeit	teils	

Die dritte Zeile ist die unbequeme: Ein Verfahren, das Verläufe auswertet, braucht die richtige der vier Zeiten — und in vielen Beständen ist nicht dokumentiert, welche gespeichert wurde. Die Datenmodellfrage aus Kapitel 2 entscheidet darüber, ob solche Auswertungen überhaupt möglich sind.

Drei Einsichten

Erstens: Der zweite Entwurf ist meist länger.

Fall	Zeilen	Verstehenszeit
Kapitel 13	+9 %	−**74** %
Kapitel 14	+14 %	−**78** %
Kapitel 4	+108 %	−77 %

Dreimal derselbe Befund: mehr Zeilen, deutlich weniger Verstehenszeit. Wer Kürze als Ziel setzt, optimiert gegen die Lesbarkeit — und Lesen ist die häufigere Tätigkeit.

Zweitens: Die Datenstruktur entscheidet.

Umgestellt auf	Folge	
Anordnungen mit Gültigkeit	**Abfrage in** 4 **Zeilen**	Kapitel 2
Ergebnistyp mit Fällen	**Absturz unmöglich**	
Ereignisse statt Liste	4 **Fehlerklassen weg**	Kapitel 6

In allen drei Fällen wurde kein Verfahren verbessert — es wurde eine Datenstruktur geändert, und das Verfahren wurde trivial oder überflüssig. Das ist Wirths Rangordnung aus Kapitel 2, dreimal bestätigt.

Drittens: Das Handwerk ist wichtiger als die Sprache.

Fertigkeit	Sprach-abhängig?	
Aufgabe verstehen	nein	Kapitel 1
Daten entwerfen	nein	
Fehler systematisch suchen	**nein**	**Kapitel 7**
Fremden Code lesen	nein	Kapitel 8
Wissen, wann es reicht	nein	
Sprachmittel kennen	ja	2 **von** 6 **Kapiteln**

Fünf von sechs Fertigkeiten sind sprachunabhängig. Das ist die Rechtfertigung dafür, dass die Bände 21 und 22 dieselben Fragen in C und Go stellen — und die Antworten sich weniger unterscheiden werden, als man erwartet.

Was dieser Band nicht behauptet

Behauptung	Aufgestellt?	
Python ist die beste Sprache	nein	**Bände 21, 22**
Es gibt beste Praktiken	**nein**	nur Bedingungen
Der zweite Entwurf lohnt immer	nein	Kapitel 4
Diese Zahlen sind Konstanten	nein	**Größen-ordnungen**

Die zweite Zeile ist die wichtigste. Jede Regel dieses Bandes hat einen Anwendungsbereich, und der steht dabei. Eine Regel ohne Ausnahmebedingung ist ein Dogma — und Dogmen erzeugen schlechten Code von der anderen Sorte.

Der erste Entwurf beantwortet die Frage,
ob man das Problem lösen kann.
Der zweite beantwortet die Frage,
ob jemand anderes die Lösung verstehen wird.

— die Bilanz dieses Bandes.
Die erste Frage stellt sich einmal.
Die zweite stellt sich bei jeder Änderung, jahrelang.

Die Sprachvorschläge[23] halten die Leitsätze fest, an denen sich Python misst; Percival und Gregory[21] zeigen die Bauformen für größere Anwendungen; Slatkin[24] führt den jeweils aktuellen Stand der Sprachmittel mit Begründung.

Part VI

Anhang

A Regeln und Kenngrößen

Die Prüffragen

Aufgabe	Welche Entscheidung soll das Programm ermöglichen?
Daten	Hält das Modell zwanzig echte Sonderfälle aus?
Zerlegung	Kann ich in einem Satz ohne „und“ sagen, was sie garantiert?
Zustand	Ist die Fachlogik ohne Umgebung prüfbar?
Lesbarkeit	Wie lange braucht ein Fremder für fünf Fragen?
Abstraktion	Ändern sich die Teile aus demselben Grund?
Aufhören	Sind die vier Bedingungen erfüllt?

Sieben Fragen, alle beantwortbar, keine davon Geschmackssache. Wer sie beim Entwerfen stellt, braucht die meisten Regeln dieses Bandes nicht auswendig.

Die vier Abbruchbedingungen

Verstehenszeit	Ein Fremder beantwortet die üblichen Fragen zügig
Änderbarkeit	Die nächste erwartete Änderung berührt **eine** Stelle
Prüfbarkeit	Die Fachlogik ist ohne Aufbau prüfbar
Größenordnung	Es hält den schwersten realen Fall aus

Die wichtigsten Zahlen

Größe	Wert	Kapitel
Zeitanteil Lesen	$\approx 30\,\%$	8
Zeitanteil Fehlersuche	$\approx 25\,\%$	7
Zeitanteil Neuschreiben	$\approx 20\,\%$	
Arbeitsgedächtnis	≈ 4 Einheiten	4
Verstehenszeit, dicht gegen benannt	Faktor ≈ 4	4
Systematisch lesen nach 8 h	$\approx 75\,\%$	8
Von vorn lesen nach 8 h	$\approx 15\,\%$	8
Fall verkleinern	12 min statt > 4 h	7
Halbierung bei n Übergaben	$\log_2 n$ Prüfungen	7
Trefferquote beim Raten	$\approx 20\,\%$	12
Zeit in Datenbankabfragen	$\approx 55\,\%$	12
Ertrag Durchgang 2 / 3 / 4	74 / 13 / 4 %	14
Günstige Funktionsgröße	15–40 Zeilen	5
Regel der drei Fälle	ab dem dritten	14

> *Alle Werte sind Größenordnungen. Die Verhältnisse sind belastbarer als die absoluten Zahlen — und auf Verhältnissen beruhen die Argumente dieses Bandes.*

B Landkarte und Glossar

Wo was steht

Kap.	Thema	Verweise
1	Die Aufgabe	**Band 18**
2	Daten zuerst	**Band 17, Kap. 4**
3	Der Ablauf	**Band 17, Kap. 9**
4	Der naheliegende Weg	Band 18, Kap. 2
5	Zerlegung	**Band 17, Kap. 4**
6	Zustand	Band 17, Kap. 5
7	Fehlersuche	**Band 2**
8	Fremden Code lesen	**Band 18**
9	Prüfen	**Band 17, Kap. 7**
10	Was Python anders macht	Band 6
11	Werkzeuge der Sprache	
12	Wo Python weh tut	**Band 1, Band 4**
13	Umschreiben	**Band 17, Kap. 13**
14	Wann aufhören	Band 17, Kap. 1
15	Stand 2026	Bände 21, 22

Begriffe

Abbruchbedingung	Prüfbares Kriterium, wann der Entwurf reicht
Anlass	Begründung für eine Warnung, statt eines Wahrheitswerts
Eigenschaftstest	Prüfung einer Regel über viele erzeugte Fälle
Erzeuger	Funktion, die Werte einzeln liefert statt gesammelt
Festhaltender Test	Friert das jetzige Verhalten ein, ohne es zu bewerten
Halbieren	Fehlerursache durch Zweiteilung eingrenzen
Kontextverwalter	Sichert Aufräumen auch bei Ausnahmen
Protokoll	Bindung an Verhalten statt an eine Hierarchie
Reine Funktion	Ergebnis hängt nur von den Argumenten ab
Tiefe Schnittstelle	Verbirgt viel, legt wenig offen
Verkleinern	Fehlerfall auf das Minimum reduzieren
Verstehenszeit	Gemessene Dauer bis zur richtigen Antwort
Zweiter Entwurf	Überarbeitung nach der ersten lauffähigen Fassung

C Kommentierte Quellen

[1] Abelson, H., Sussman, G.: *Structure and Interpretation of Computer Programs*. 2. Aufl., MIT Press 1996.

Kein Buch über eine Sprache, sondern über das Denken beim Programmieren. *Zeigt, wie sich große Systeme aus wenigen Bausteinen aufbauen lassen. Anspruchsvoll, zeitlos, frei verfügbar.*

[2] Beazley, D., Jones, B.: *Python Cookbook*. 3. Aufl., O'Reilly 2013.

Rezeptsammlung mit ausführlichen Begründungen — die Erläuterungen sind wertvoller als die Rezepte. In Einzelheiten veraltet, in der Denkweise nicht.

[3] Beck, K.: *Implementation Patterns*. Addison-Wesley 2007.

Über die kleinen Entscheidungen: Namen, Sichtbarkeit, Zerlegung. Dünn, unterschätzt und näher an der täglichen Arbeit als die meisten Entwurfsbücher.

[4] Bentley, J.: *Programming Pearls*. 2. Aufl., Addison-Wesley 1999.

Das beste Buch über den Weg vom Problem zur Lösung. *Jede Kolumne beginnt mit einer Aufgabe, geht den naheliegenden Weg und entwickelt daraus den besseren — genau die Methode dieses Bandes.*

[5] Bentley, J.: *More Programming Pearls*. Addison-Wesley 1988.

Die Fortsetzung, mit stärkerem Gewicht auf Messung und Fehlersuche. Der Aufsatz über die Rückverfolgung eines Leistungsproblems ist ein Lehrstück.

[6] Dijkstra, E.: *A Discipline of Programming*. Prentice Hall 1976.

Entwickelt Programme aus ihrer Angabe. Für die tägliche Arbeit sperrig — die Invariante als Werkzeug des Verstehens stammt von hier und ist unverzichtbar.

[7] Fowler, M.: *Refactoring*. 2. Aufl., Addison-Wesley 2018.

Der Katalog der Umgestaltungsschritte, jeweils mit Anlass und Gegenanzeige. Der wichtigste Satz steht früh: ohne Prüfnetz ist es keine Umgestaltung.

[8] Gamma, E., u. a.: *Design Patterns*. Addison-Wesley 1994.

Sein Wert liegt im Vokabular, nicht im Katalog. In Python sind mehrere der Muster überflüssig, weil die Sprache sie eingebaut hat — das zu erkennen ist Teil der Kunst.

[9] Hettinger, R.: Vorträge zu idiomatischem Python. 2013 ff.

Frei verfügbare Aufzeichnungen, die den Unterschied zwischen möglich *und* angemessen *zeigen. Wer wissen will, was pythonisch heißt, findet hier mehr als in jedem Buch.*

[10] Hunt, A., Thomas, D.: *The Pragmatic Programmer*. 2. Aufl., Addison-Wesley 2019.

Über die Haltung zum Handwerk — Nachfragen, Werkzeuge beherrschen, Verantwortung übernehmen. Stellenweise sentenzenhaft, in der Summe sehr nützlich.

[11] Kernighan, B., Pike, R.: *The Practice of Programming*. Addison-Wesley 1999.

Zweihundert Seiten Handwerk ohne eine Zeile Methodik. *Fehlersuche, Prüfung, Werkzeuge, Portabilität — praktisch nützlicher als vieles, was seither erschienen ist.*

[12] Kernighan, B., Plauger, P.: *The Elements of Programming Style*. 2. Aufl., McGraw-Hill 1978.

Nimmt echte Programme auseinander und schreibt sie um. Die Sprache ist Fortran und PL/I; die Regeln gelten unverändert. Das Vorbild für die Methode dieses Bandes.

[13] Knuth, D.: *Literate Programming*. CACM 27(2), 1984.

Fordert, Programme für menschliche Leser zu schreiben und den Übersetzer als zweitrangig zu behandeln. Das Werkzeug hat sich nicht durchgesetzt, die Forderung ist geblieben.

[14] Knuth, D.: *The Art of Computer Programming*. Addison-Wesley, seit 1968.

Das Referenzwerk zu Verfahren und ihrer Analyse. Kein Buch zum Durchlesen; zum Nachschlagen unersetzlich. Von hier stammt auch die Warnung vor verfrühter Optimierung, meist verkürzt zitiert.

[15] Martelli, A., u. a.: *Python in a Nutshell*. 4. Aufl., O'Reilly 2023.

Die gründlichste Referenz mit Erläuterungen zum Laufzeitverhalten. Zum Nachschlagen, wenn man wissen will, was tatsächlich geschieht.

[16] McConnell, S.: *Code Complete*. 2. Aufl., Microsoft Press 2004.

Umfassend und mit Belegstellen für jede Empfehlung. Schlagen Sie nach, statt es durchzulesen — die Kapitel zu Namensgebung und Zerlegung sind die besten ihrer Art.

[17] Norvig, P.: Aufsätze und Programmierstudien. Fortlaufend.

Frei verfügbare, vollständig durchgearbeitete Lösungen zu nichttrivialen Aufgaben — mit allen Zwischenschritten und Verwerfungen. Die beste Sammlung gelöster Beispiele, die es im Netz gibt.

[18] Okken, B.: *Python Testing with pytest*. 2. Aufl., Pragmatic Bookshelf 2022.

Praxisnah und knapp. Behandelt auch das, was meist fehlt: wie man Prüfungen so schreibt, dass sie Umbauten überleben.

[19] Ousterhout, J.: *A Philosophy of Software Design*. Yaknyam Press 2018.

Hundertneunzig Seiten gegen die Zergliederung. *Fordert tiefe Schnittstellen und widerspricht ausdrücklich der Regel, Funktionen möglichst klein zu halten. Die Auseinandersetzung damit lohnt.*

[20] Parnas, D.: *On the Criteria To Be Used in Decomposing Systems into Modules*. CACM 15(12), 1972.

Fünf Seiten, die den Modulbegriff begründet haben. Stellt zwei Zerlegungen gegenüber und zeigt, dass die naheliegende die schlechtere ist. In zwanzig Minuten gelesen, danach zerlegt man anders.

[21] Percival, H., Gregory, B.: *Architecture Patterns with Python*. O'Reilly 2020.

Führt die Trennung von reiner Fachlogik und zustandsbehafteten Rändern konsequent durch. Stellenweise überkonstruiert — die Grundidee trägt und ist in Kapitel 6 übernommen.

[22] Ramalho, L.: *Fluent Python*. 2. Aufl., O'Reilly 2022.

Das beste Buch über Python für Fortgeschrittene. *Erklärt nicht die Syntax, sondern das Datenmodell dahinter — und damit, warum die Sprache sich verhält, wie sie sich verhält.*

[23] Python-Sprachvorschläge: PEP 8 (Stil), PEP 20 (Leitsätze), PEP 484 (Typangaben).

Kurz, verbindlich und frei verfügbar. PEP 20 passt auf eine Seite und enthält mehr Entwurfslehre als manches Lehrbuch — besonders der Satz, dass Fehler niemals stillschweigend vergehen sollen.

[24] Slatkin, B.: *Effective Python*. 3. Aufl., Addison-Wesley 2024.

Einzelregeln mit Begründung und Gegenbeispiel. Die geeignete Ergänzung zu Ramalho: dieser erklärt das System, jener die täglichen Entscheidungen.

[25] Spinellis, D.: *Code Reading*. Addison-Wesley 2003.

Das einzige Buch über das Lesen von Code *— und damit über die Tätigkeit, die dreißig Prozent der Zeit ausmacht und nirgends gelehrt wird. Die Beispiele sind C; die Verfahren sind allgemein.*

[26] Spinellis, D.: *Code Quality*. Addison-Wesley 2006.

Der Folgeband, über die Eigenschaften, an denen man Qualität erkennt und misst. Nüchtern und ohne Bekenntnisse — eine Seltenheit in diesem Bereich.

[27] Wirth, N.: *Algorithms + Data Structures = Programs*. Prentice Hall 1976.

Der Titel ist die These, und sie enthält eine Rangordnung. *Wer die Datenstruktur richtig wählt, bekommt das Verfahren fast geschenkt. Das ist die tragende Einsicht von Kapitel 2.*

[28] Zeller, A.: *Why Programs Fail*. 2. Aufl., Morgan Kaufmann 2009.

Macht aus der Fehlersuche ein Verfahren *— mit Hypothesen, widerlegenden Vorhersagen und systematischem Verkleinern. Das einzige Buch, das diese Tätigkeit ernst nimmt, und die Grundlage von Kapitel 7.*

Wenn Sie nur drei Bücher kaufen

[4]	**Bentley**	Für den Weg vom Problem zur Lösung
[22]	**Ramalho**	Für Python, wie es gemeint ist
[28]	**Zeller**	**Für die Fehlersuche als Verfahren**

Bentley lehrt das Denken, Ramalho die Sprache, Zeller das Handwerk. Auffällig ist, was fehlt: kein Buch über eine Methodik und keines über Muster — beides ist zweitrangig gegenüber der Fähigkeit, ein Problem zu durchdringen.

Zwei Ergänzungen: [20] Parnas, fünf Seiten in zwanzig Minuten, die die Zerlegung verändern. Und [25] Spinellis — weil Lesen die häufigere Tätigkeit ist und es dazu genau ein Buch gibt.